五一数学建模竞赛

承　诺　书

我们仔细阅读了五一数学建模竞赛的竞赛规则。

我们完全明白，在竞赛开始后参赛队员不能以任何方式（包括电话、电子邮件、网上咨询等）与本队以外的任何人（包括指导教师）研究、讨论与赛题有关的问题。

我们知道，抄袭别人的成果是违反竞赛规则的，如果引用别人的成果或其它公开的资料（包括网上查到的资料），必须按照规定的参考文献的表述方式在正文引用处和参考文献中明确列出。

我们郑重承诺，严格遵守竞赛规则，以保证竞赛的公正、公平性。如有违反竞赛规则的行为，我们愿意承担由此引起的一切后果。

我们授权五一数学建模竞赛组委会，可将我们的论文以任何形式进行公开展示（包括进行网上公示，在书籍、期刊和其他媒体进行正式或非正式发表等）。

参赛题号（从 A/B/C 中选择一项填写）：A

参赛队号：T3836544926041

参赛组别（研究生、本科、专科、高中）：本科

所属学校（学校全称）：成都信息工程大学

参赛队员：队员 1 姓名：梁海洲

队员 2 姓名：王佳璐

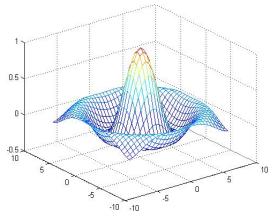
队员 3 姓名：袁欣

联系方式：Email：lianghz1022@163.com 联系电话 18030702904

日期：2024 年 5 月 3 日

（除本页外不允许出现学校及个人信息）

五 一 数 学 建 模 竞 赛



题 目：基于改进模拟退火算法的钢材最优切割方案研究

关键词：图论　TSP 问题　模拟退火算法　贪心思想

摘　要：

对于问题一，要为下料切割布局 N1 设计最优切割路径方案，并给出最优切割路径的空程总长度。本文首先将 N1 依据图论的基本理论，划分出它的点，边和权值，然后将其转化为一个**旅行商（TSP）问题**，从点 B_1 出发，以最短路径访问所有的点一次且仅一次，并回到点 B1，然后用**模拟退火算法**求解得到结果，发现此时规划的切割路径缺少 B3 到 B4 这条边，因此在 B3 处增加一个点，从侧面解决切割路径 B3-B4 缺失的问题，再用模拟退火算法求得结果，最后发现由于算法的局限性不能很好的按照直线上所有的点进行空程计算，再根据等底等高的三角形另外两边之和最短时是等腰三角形进一步优化。最优切割路径为 **B1-B2-B3-A3-A4-A1-A2-A3 和 A4 的中点-B4-B1**，最短空程为 **64.0312**。

对于问题二，要设计下料切割布局 N2 的最优切割路径方案，并给出最优切割路径的空程总长度。本文着重研究**先沿锯齿状切割线切割完后再切割内部图形**的方案，将所有特征点标号后，沿用问题一中改进后的模拟退火算法通过 Matlab 软件求得最优切割路径见图 10，最短空程为 **31.5535**。

对于问题三，要设计下料切割布局 N3 的最优切割路径方案，并给出最优切割路径的空程总长度。通过观察比较发现，N3 布局是在 N2 布局上在椭圆切割线的内部增加了 12 个小矩形，根据**贪心算法的核心思想**，先求得椭圆和小矩形的局部最优解，再加上问题二中的最优解，组合起来即全局最优解。因此本问中仅针对椭圆及其内部的小矩形上的特征点进行标号，并沿用问题一中改进后的模拟退火算法求得最优切割路径，最优切割路径见图 14，最短空程为 **78.5215**。

对于问题四，要设计下料切割布局 N4 的最优切割路径方案，并给出最优切割路径的空程总长度。首先对图中要用到的特征点标号，再依据 A43 到椭圆的距离是否大于 3 制定出两种不同的“过桥”方案。但经过计算发现，题目所给的需要添加过桥的嵌套矩形件套料图 N4 和图中所给的数据存在矛盾，为了模拟图形的正常切割，本文根据题目所给数据画出正确的 N4 下料切割布局，并假设在椭圆外部的 A43 点在椭圆圆周上。此时本问中 A43 到椭圆的距离为 0，小于 3，故本文选择将四个矩形小零件切割成两个通过“过桥”连接的大零件的方案。最后利用 Matlab 软件由改进条件后的模拟退火算法求解，得到最优切割路径方案见图 18，最短空程为 **43.5535**。

一、 问题重述

1.1 问题背景

模具加工是指成型和制坯工具的加工，此外还包括剪切模和模切模具。剪切模具的过程就是下料，即从整个材料中取下一定形状、数量的材料的操作过程。钢板切割就是根据提前绘制好的下料切割布局图纸对钢板进行加工，该切割路径的设计尤为重要，直接关系到钢板下料切割过程中的工作效率，是模具加工企业降低成本和增加经济效益的重要途径。

1.2 问题重述

建立数学模型，分别设计对 N1~N4 四个下料切割布局的最优切割路线，并且计算出最优切割路径的空程长度：

下料切割布局 N1：B3-B4 是钢板边界线（不用切割），B1 是切割起点。

下料切割布局 N2：钢板的外边界需切割成上下对称的锯齿状，钢板上部需切割出四个小圆，中部需切割出一个椭圆，右下角点是切割起点。

下料切割布局 N3：钢板的外边界需切割成上下对称的锯齿状，钢板上部需切割出四个小圆，中部需切割出一个椭圆，同时椭圆内部还需切割出同等大小的十二个矩形，右下角点是切割起点。

下料切割布局 N4：钢板的外边界需切割成上下对称的锯齿状，钢板上部需切割出四个小圆，中部需切割出一个椭圆，同时椭圆内部还需切割出四个矩形，右下角点是切割起点。为防止小零件掉落，相邻的两个小零件之间需要采用“过桥”的方式使其链接在一起，需要给出“过桥”的数目和位置。

二、 问题分析

2.1 问题一的分析

问题一要设计下料切割布局 N1 的最优切割路径方案，本文先将 N1 依据图论的基本理论，将两个矩形的 4 个顶点共 8 个顶点作为点，矩形的 4 条边共 7 条边（切割边界线除外）作为边，各边的实际长度作为该边的权值。然后将其转化为一个旅行商（TSP）问题，从 B1 点出发，以最短路径访问所有的点一次且仅一次，并回到 B1 点。再用模拟退火算法求解得到结果，发现此时规划的切割路径少了 B3 到 B4 这条边，因此在 B3 处增加一个点，从侧面解决切割路径 B3-B4 缺失的问题，再用模拟退火算法求得结果。最后发现由于算法的局限性不能很好的按照直线上所有的点进行空程计算，再根据等底等高的三角形另外两边之和最短时是等腰三角形进一步优化。

2.2 问题二的分析

问题二要设计下料切割布局 N2 的最优切割路径方案，经过分析，切割方案可以分为两种；方案一可以沿锯齿状切割线切割时同时选取最佳时机切割内部图形，方案二是先沿锯齿状切割线切割完后再切割内部图形。显然，方案一的空程会更短，因此本问着重研究方案一的切割路径。为了方便计算和表示，首先需要对题中所给图形的顶点处标号，有三类地方需要标号，分别是：①将图中所有拐点处标号；②沿用问题一的思想，对所有组成需要折返路径的两个端点中位于锯齿状切割线上的点处新增一

个点并分别标号 A46-A50；③寻找内部小圆和椭圆上距离锯齿状切割线上最近的点并标号。然后将问题一中改进后的算法应用到本问中。

2.3 问题三的分析

问题三要设计下料切割布局 N3 的最优切割路径方案，通过观察比较发现，N3 布局仅是在 N2 布局上在椭圆切割线的内部增加了 12 个小矩形，根据贪心算法的思想，先求得椭圆和小矩形的局部最优解，再加上问题二中的最优解，合起来就是全局最优解。

本文首先设椭圆上距离右上角矩形的右上角顶点最近的点为特征点 A1，为了确保算法中每一个切割线都能准确被切割，将椭圆内小矩形的所有点全部标号两次，同样，表示同一位置的两个点之间的距离设为无穷。依据从下往上，从右往左的顺序排列 12 个小矩形，每个小矩形的四个顶点按照图的命名规则分别标号，共计 97 个特征点。其次将问题一中改进后的算法应用到本问中，最后再把此时得到的最优解和问题二中的最优解结合起来就是最终的最优切割路径方案。

2.4 问题四的分析

问题四要设计下料切割布局 N4 的最优切割路径方案，先对图中要用到的特征点进行标号：①将图中所有拐点处标号；②沿用问题一的思想，对组成需要折返路径的两个端点中位于锯齿状切割线上的那个点处新增一个点并标号；③寻找内部小圆和椭圆上距离锯齿状切割线上最近的点并标号；④为“过桥”的端点标号。

设四个矩形小零件距离椭圆最近的点分别为 A，B，C，D，其与椭圆的距离分别为 a，b，c，d（a>b>c>d）。本文经过定性分析发现，问题四的广义最优“过桥”方案共有 2 种：①当 c 大于两个相邻小零件距离的最小值时，最优切割方案为从 D 处对小零件进行切割，切割结果为 2 个大零件或者 1 个大零件，此时“过桥”数为 2 或 3。②反之，最优切割方案为从 D 处出发切割 2 个小零件为 1 个大零件后返回椭圆，再沿椭圆到达 C 处，后对余下 2 个小零件切割使之成为 1 个大零件。此时切割结果为 2 个大零件，“过桥”数为 2。

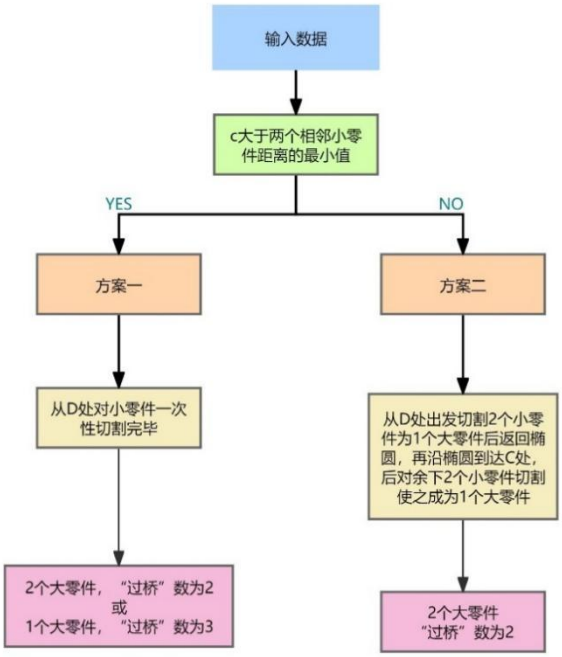


图 1 切割方案判断流程图

针对本问具体数据，本文经过计算发现，题目所给的需要添加过桥的嵌套矩形件套料图 N4 和图中所给的数据存在矛盾，点 A56 应在椭圆圆周上，点 A43 应在椭圆外，并非题目图中所画的均在椭圆内部。为了模拟图形的正常切割，本文根据题目所给数据画出正确的 N4 下料切割布局，并假设在椭圆外部的 A43 点在椭圆圆周上。

根据假设，此时 A43 到椭圆的距离为 0，小于 3，因此本问在计算时选择切割方案②，此时大零件的个数应为 2，“过桥”数应为 2。最后利用 Matlab 软件由问题一的模拟退火算法添加判断条件后求解，得到最优切割路径方案和最短空程。

三、 模型假设

假设 1：内孔零件的切割方式为一次性沿着闭合回路完成切割：为了简化模型且方便实际切割中的操作，将每一个内孔零件看做一个闭合回路，每次切割都是一次性沿着闭合回路完成该零件的切割。

假设 2：表示同一位置的 2 个特征点距离为无穷：TSP 问题要求每个点只能经过一次，但是在实际计算时发现如果仅经过一次有的切割路径就会缺失，因此为了不违背算法又解决切割路径缺失的问题，本文把同一个位置标记两次，且标记的两个点之间的距离是无穷。

假设 3：问题四中在代入模拟退火算法时点 A56 和点 A43 均在圆周上：根据题中所给数据算出，点 A56 应在椭圆圆周上，点 A43 应在椭圆外，并非题中所给图上的两个点均在椭圆内部，考虑实际切割误差，假设两个点均在椭圆圆周上，做近似处理。

四、 符号说明

符号	说明
x_{ij}	是否经过v_i和v_j之间的路径
d_{ij}	点v_i到点v_j之间的距离
z	空程的长度
Δf	后一条路径与前一条路径长度之差
T	初始温度
p	接受新解的概率

五、 模型的建立与求解

5.1 问题一模型的建立与求解

5.1.1 绘制 N1 的下料切割示意图

对于一个图论问题，首先需要确定图中的顶点、边及路径上的权值。在本问中，我们将 2 个矩形的 4 个顶点，共 8 个点作为图中的顶点；将切割线和钢板边界线作为图中的边；将每条边的实际长度作为该边的权值。基于以上思想，建立以下下料切割示意图，如图 2 所示。

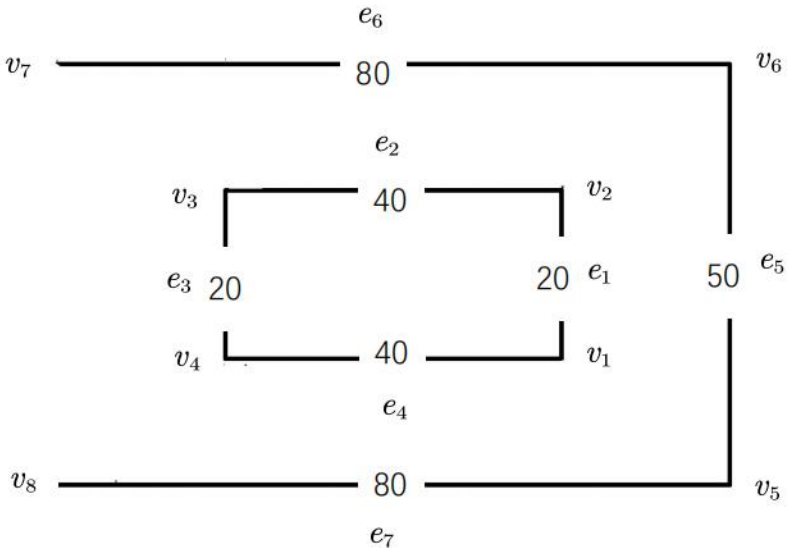


图 2 N1 下料切割示意图

5.1.2 N1 最优切割路径模型的建立

对于切割路径的优化，实质上是一个旅行商（TSP）问题，从 B1 点出发，以最短路径访问所有的点一次且仅一次，并回到 B1 点。由于题目要求要使空程最短，因此是一个有目标函数的旅行商问题。

（1）决策变量

切割路径的优化要明确的给出每一条路径是否需要经过，而每一条路径又是由两个点来确定的，所以最终的决策变量就是两点确定的路径是否需要经过。

$$x_{ij}=\begin{cases}1,\text{要经过}v_i\text{和}v_j\text{之间的路径}\\0,\text{否则}\end{cases}\quad(i,j=1,2,\cdots,8)\tag{1}$$

（2）目标函数

题目要求最优切割路径要满足空程最短的原则，即总的路径再减去有效切割的路径后要最短，得到的目标函数为：

$$\min\ z=\sum\sum x_{ij}\times d_{ij}(i,j=1,2,\cdots,8)-330\tag{2}$$

其中，d_{ij}表示点v_i到点v_j之间的距离，即e_1，e_2，…，e_7的长度，330 是有效切割长度之和。

（3）约束条件

在求解最优路径的时候，因为用到了 TSP 问题的模型，因此要考虑到这个问题本身的一些约束条件，每个点被到达且仅被到达一次，每个点离开且只被离开一次。

5.1.3 N1 最优切割路径模型的求解

算法：求解目标函数采用的模拟退火算法[1]如下。

（1）解空间

解空间S可表示为$\{1,2,\cdots,8\}$的所有仅固定起点为 5 的循环排列组合，其中每一个循环排列表示遍历 8 个目标点的一个回路，随机生成初始解。

（2）目标函数

此时的目标函数为便利所有目标的路径长度或称代价函数，即

$$\min\ z=\sum\sum x_{ij}\times d_{ij}(i,j=1,2,\cdots,8)-330 \tag{1}$$

（3）新解的产生

1、交换法

任选序号$u,v(u<v)$交换u和v之间的顺序，此时的新路径为：

$$e_1\cdots e_{u-1}e_v e_{u+1}\cdots e_7 \tag{2}$$

2、位移法

任选序号u，v和$w(u<v<w)$，将u，v之间的路径插到w之后，此时的新路径为：

$$e_1\cdots e_{u-1}e_v e_{u+1}\cdots e_7 \tag{3}$$

3、倒置法

任选序号u，v，如果u大于v，则将u和v之间（包括u，v）的序号倒置，此时的新路径为：

$$e_1\cdots e_{u-1}e_{v+1\cdots}e_w e_u\cdots e_v e_{w+1}\cdots e_7 \tag{4}$$

随机生成一个 0 到 1 之间的随机数s，若$0<s<0.33333$，则选择交换法产生新解，若$0.3333<s<0.6666$。则选择位移法产生新解，其余则选择倒置法产生新解。

（4）代价函数差

路径差Δf是后一条路径与前一条路径长度之差，可表示为：

$$\Delta f=z_{u+1}-z_u \tag{5}$$

（5）接受准则

$$p=\begin{cases}1, & \Delta f<0\\ \exp(-\Delta f/T), & \Delta f\geqslant 0\end{cases} \tag{6}$$

其中，本问设定的初始温度T为 1000。如果$\Delta f<0$，则接受新的路径；否则以概率$\exp(-\Delta f/T)$接受新的路径，即当$\exp(-\Delta f/T)$的值大于 0 到 1 的随机数则接受。

（6）降温

利用选定的降维系数$\alpha=0.95$进行降温，即$T\leftarrow\alpha T$，得到新的温度。

（7）终止条件

用迭代次数作为终止条件，判断退火过程是否结束，若迭代次数大于等于 1000，算法结束，退出当前状态。

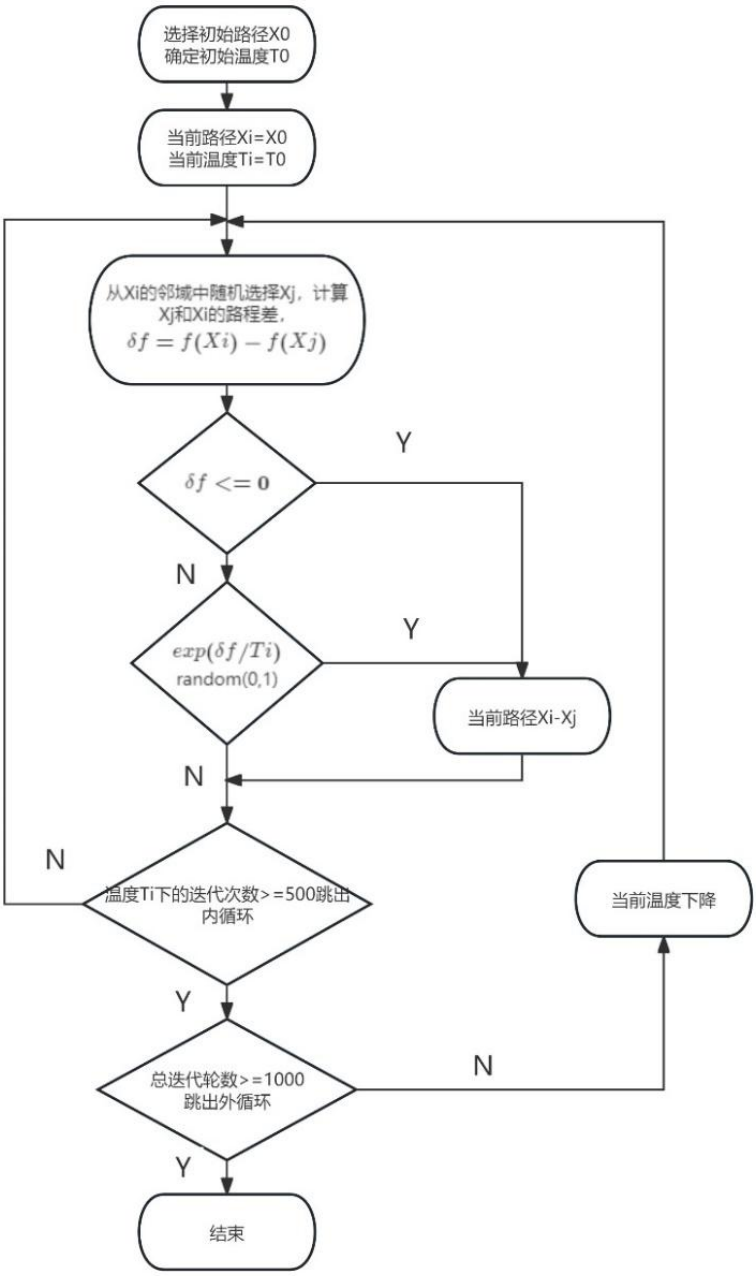


图 3 模拟退火算法流程图

根据模拟退火算法在 Matlab 中编程计算可得，有一条最优切割路径且同时满足空程最短原则。此时解出的最优切割路径为：

$v_5 \to v_6 \to v_7 \to v_3 \to v_2 \to v_1 \to v_4 \to v_8 \to v_5$，最短空程为 50。

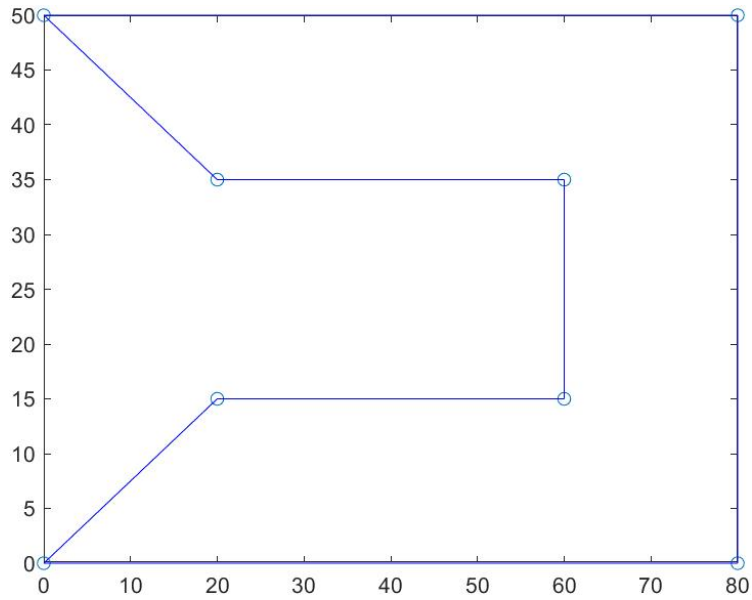


图 4 N1 切割路径方案图

从图 4 可以看出，虽然是一个最优化的切割方案，但却显然少了v_3到v_4这一切割路径，因此本问对模型加以改进，在v_3处多增加一个点v_9，即允许在规划路径时出现折返的情况。

5.1.4 N1 最优切割路径模型的改进

将模型改对 TSP 问题的模型加以改进，在v_3处多增加一个点v_9，即允许在规划路径时出现折返的情况，而v_3到v_9和v_9到v_3之间的距离都是无穷，意味着v_3和v_9之间不会出现路径，而后重新计算得到的切割方案如图 5 所示，此时的最优切割路径为

$v_5 \to v_6 \to v_7 \to v_9 \to v_4 \to v_1 \to v_2 \to v_3 \to v_8 \to v_5$，即

B1-B2-B3-A3-A4-A1-A2-A3-B4-B1，最短空程为 65.3113。

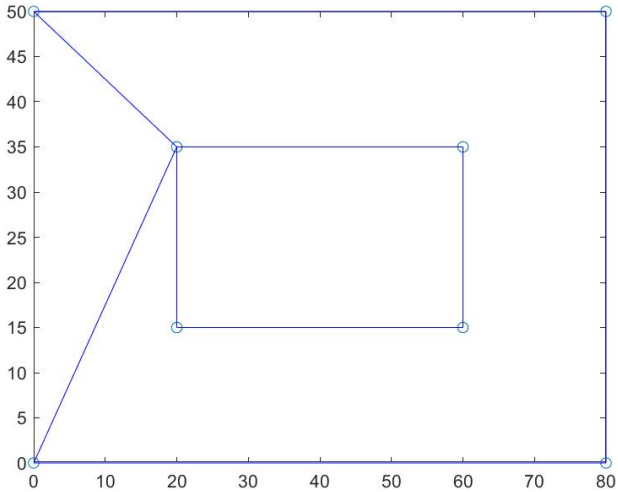


图 5 N1 切割路径方案图（改进后）

由于算法的局限性不能很好的按照直线上所有的点进行空程计算，本文发现在图 5 下还可以继续优化两条空程路径，因为等底等高的三角形另外两边之和最短时是等腰三角形，所以最后结果如图 6 所示：

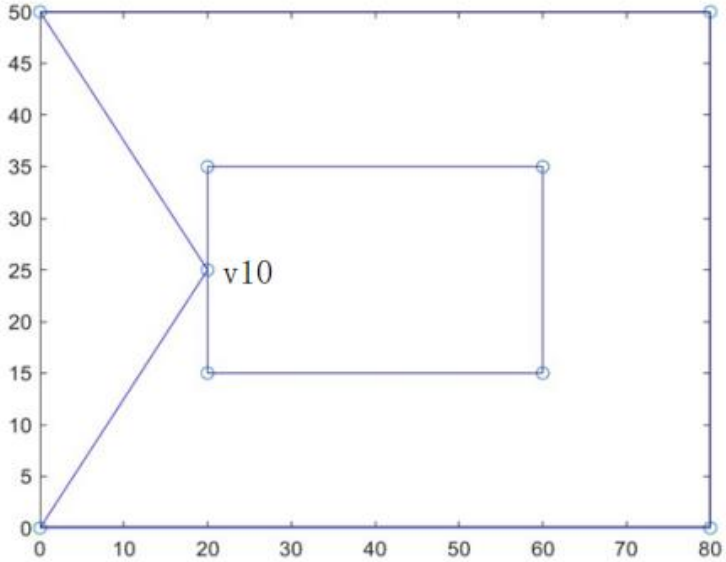


图 6 N1 切割路径最优方案图

此时的最优切割路径为$v_5 \to v_6 \to v_7 \to v_9 \to v_4 \to v_1 \to v_2 \to v_{10} \to v_8 \to v_5$，即 B1-B2-B3-A3-A4-A1-A2-(A3 与 A4 的中点)-B4-B1，最短空程为 64.0312。

5.2 问题二模型的建立与求解

5.2.1 切割方案的选取

方案一：沿锯齿状切割线切割时同时选取最佳时机切割内部图形。

方案二：先沿锯齿状切割线切割完后再切割内部图形。

显然，方案一的空程会更短，因此本问着重研究方案一的切割路径。

5.2.2 绘制 N2 的下料切割示意图

1、将图中所有拐点处标号。

2、沿用问题一的思想，对组成需要折返路径的两个端点中位于锯齿状切割线上的那个点处新增一个点并标号 A46-A50。

3、寻找内部小圆和椭圆上距离锯齿状切割线上最近的点并标号。

因为该椭圆与临近的矩形边位置对称，因此到 ABCD 四个点的路径相同，由于在 AB 直线上作到椭圆最短路径为垂线，由垂线向两边延长距离变大，到 A 点指向椭圆切线时距离最短。以左下角为原点，建立直角坐标系，Matlab 代入坐标数值计算得到该直线上的点到椭圆最短距离为 7.78，该点的坐标为（66.06,21.70）。

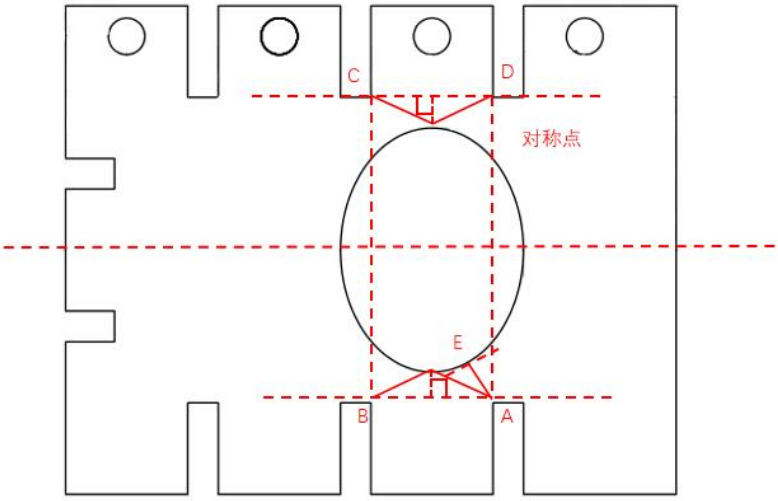


图 7 椭圆与边界距离示意图

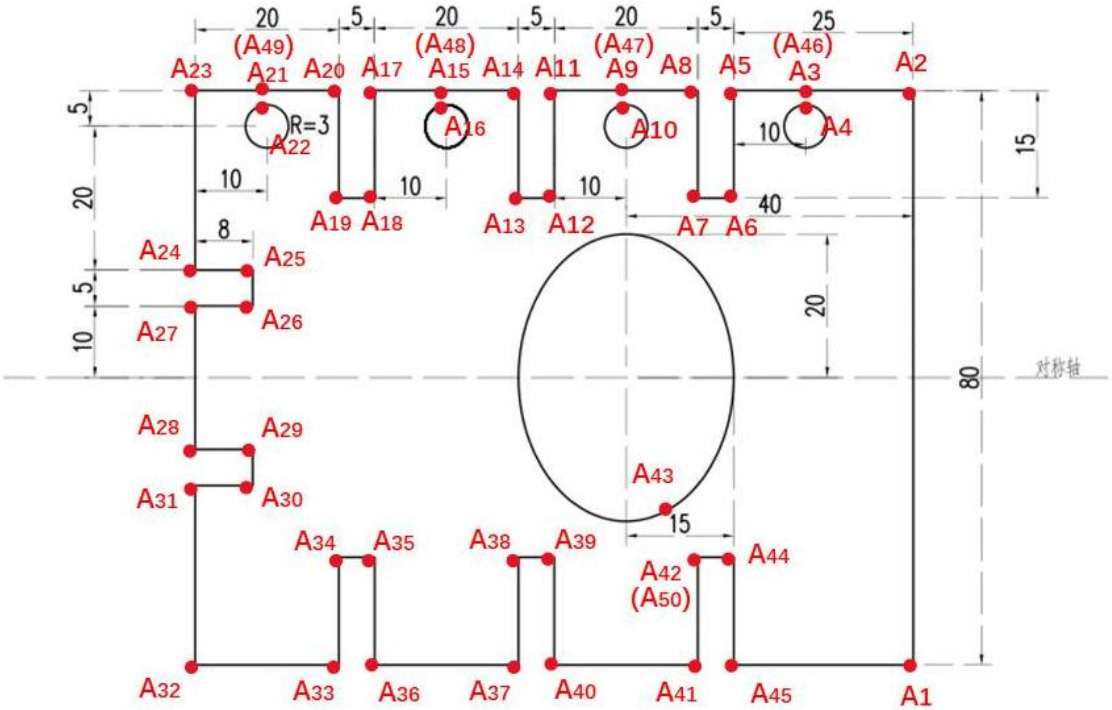


图 8 N2 切割下料示意图

最终的切割下料示意图如图 8 所示。

5.2.3 N2 最优切割路径模型的建立

（1）决策变量

切割路径的优化要明确的给出每一条路径是否需要经过，而每一条路径又是由两个点来确定的，所以最终的决策变量就是两点确定的路径是否需要经过。

$$x_{ij}=\begin{cases}1,\text{要经过}A_i\text{和}A_j\text{之间的路径}\\0,\text{否则}\end{cases}\quad(i,j=1,2,\cdots,50)\tag{1}$$

（2）目标函数

题目要求最优切割路径要满足空程最短的原则，即总的路径再减去有效切割的路径后要最短，得到的目标函数为：

$$\min\ z=\sum\sum x_{ij}\times d_{ij}(i,j=1,2,\cdots,50)-(592+54\Pi) \tag{2}$$

其中，d_{ij}表示点A_i到点A_j之间的距离，592+54π是有效切割长度之和。

（3）约束条件

1.每个点被到达且仅被到达一次，每个点离开且只被离开一次。

2.A46 和 A3，A47 和 A9，A48 和 A15，A49 和 A21，A50 和 A42 这五组点中每组中的两个点距离为无穷大，即每组点之间不会有切割路径。

5.2.4 N2 最优切割路径模型的求解

N2 中有多个封闭图形，切割该封闭图形可不考虑具体轮廓，从轮廓中提取临近点代替该轮廓，该点表示切割时这个封闭图形的起始点与终点，钢板切割点采用最邻近算法确定[2]。

利用 Matlab 软件由问题一的模拟退火算法可以解得最优的切割路径为 A1-A2-A3-A4-A46-A5-A6-A7-A8-A9-A10-A47-A11-A12-A13-A14-A15-A16-A48-A17-A18-A19-A20-A21-A22-A49-A23-A24-A25-A26-A27-A28-A29-A30-A31-A32-A33-A34-A35-A36-A37-A38-A39-A40-A41-A42-A43-A50-A44-A45-A1，最短空程为 31.5535。实际切割路径见图 10。

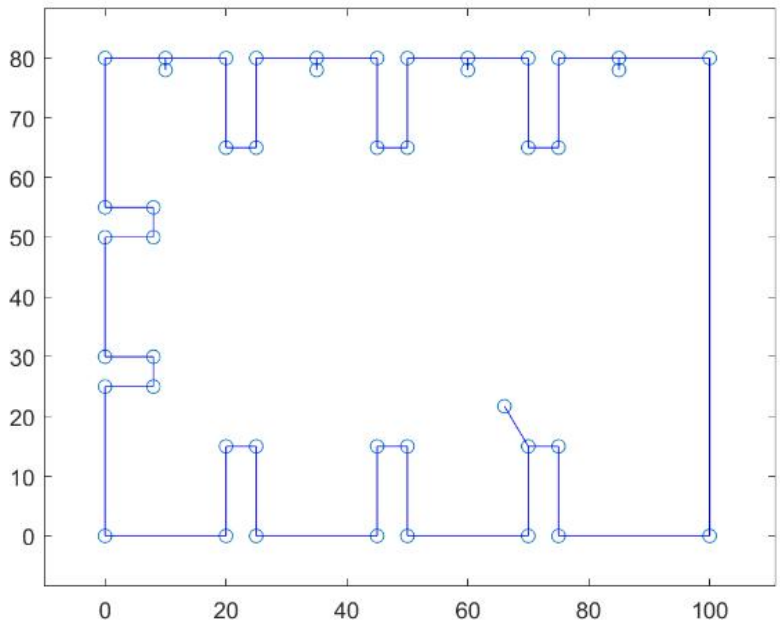


图 9 N2 切割路径最优方案图

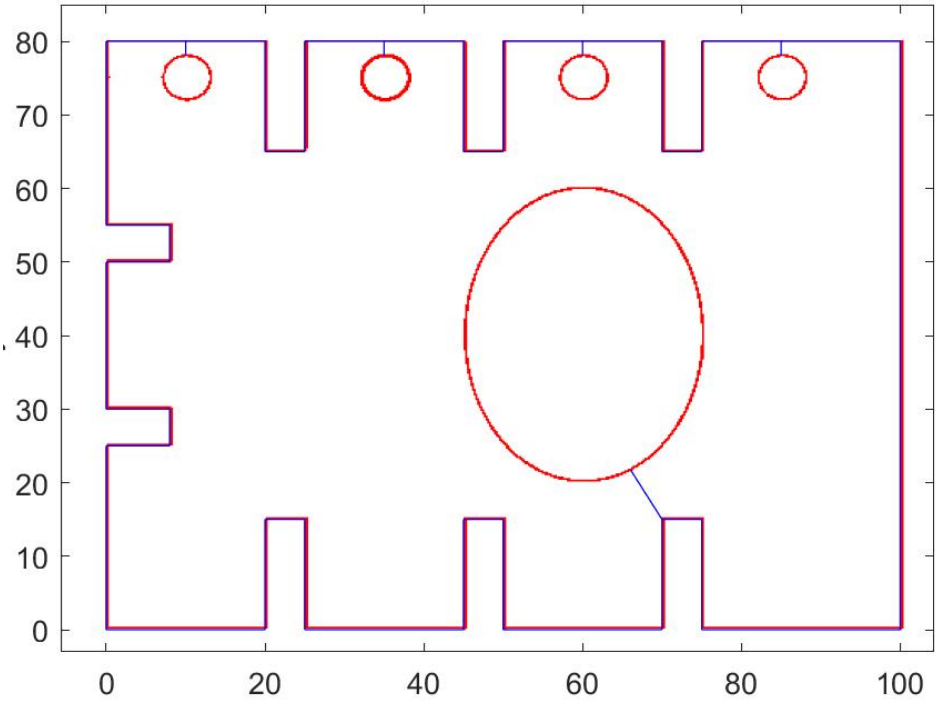
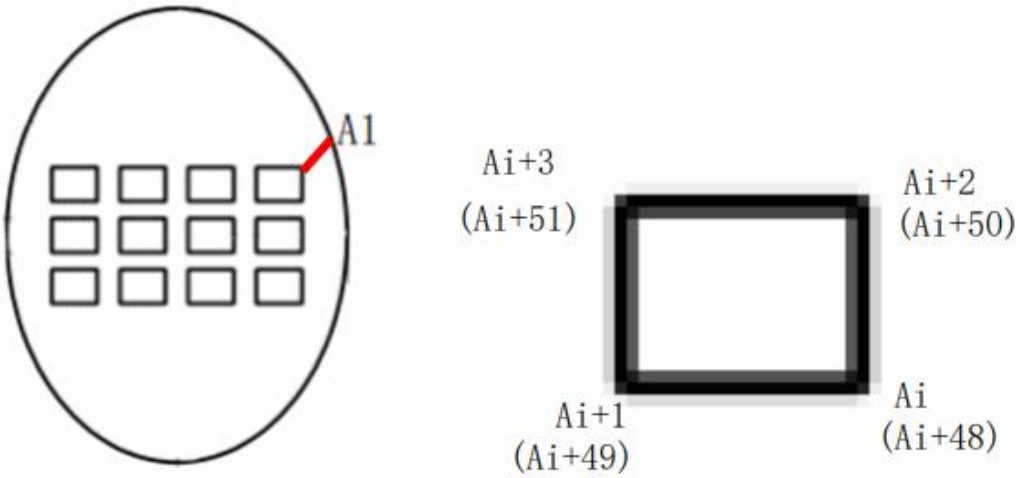


图 10 N2 实际切割路径图

5.3 问题三模型的建立与求解

5.3.1 绘制 N3 的下料切割示意图

N3 的示意图是在 N2 的基础上新增了椭圆内部的 12 个小矩形，以椭圆的中心为原点建立直角坐标系，设椭圆上距离右上角矩形的右上角顶点最近的点为特征点 A1（13.94，7.38），该点到矩形顶点的距离为 3.07，如图 11（左）所示，依据从下往上，从右往左的顺序排列 12 个小矩形。为了确保算法中每一个切割线都能准确被切割，将椭圆内小矩形的所有点全部标号两次，每个小矩形中特征点的命名规则按图 11（右）所示，其中 i 的取值为 2,6,10，……，4n-2,……，46，共计特征点 97 个。



（左）　　（右）

图 11 N3 的下料切割示意图

5.3.2 N3 最优切割路径模型的建立

贪心算法总是做出当前看来是最好的选择，从问题的某一初始解出发，同时能朝给定的总目标进一步求解，求出可行解的一个解元素，由所有解元素组合成问题的一个可行解[3]。根据贪心算法的思想，在问题二全局最优解的基础上，寻找椭圆内矩形的切割方案的最优解。

（1）决策变量

切割路径的优化要明确的给出每一条路径是否需要经过，而每一条路径又是由两个点来确定的，所以最终的决策变量就是两点确定的路径是否需要经过。

$$x_{ij}=\begin{cases}1,\text{要经过}A_i\text{和}A_j\text{之间的路径}\\0,\text{否则}\end{cases}\quad(i,j=1,2,\cdots,97)\tag{1}$$

（2）目标函数

题目要求最优切割路径要满足空程最短的原则，即总的路径再减去有效切割的路径后要最短，得到的目标函数为：

$$\min\ z=\sum\sum x_{ij}\times d_{ij}(i,j=1,2,\cdots,97)-168\tag{2}$$

其中，d_{ij}表示点A_i到点A_j之间的距离，168 是有效切割长度之和。

（3）约束条件

1. 每个点被到达且仅被到达一次，每个点离开且只被离开一次。

2. $d_{i(i+48)}=\infty(i=2,3,\cdots,49)$，即每组$A_i$和$A_{i+48}$点之间不会有切割路径。

5.3.3 N3 最优切割路径模型的求解

沿用问题二中的最邻近算法思想，利用 Matlab 软件由问题一的模拟退火算法求解，发现由于在算法中设置的随机性，每次的结果不尽相同，但如图 12 所示，随着迭代次数的增加，最短路径的和值逐渐收敛于固定值，这个值就是模拟退火算法最优值。

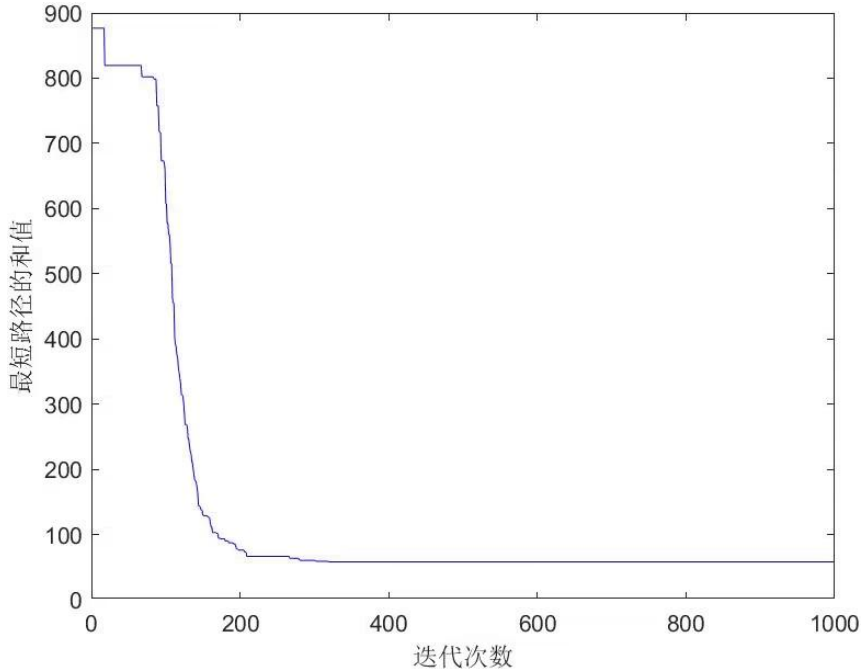


图 12 N3 局部最短空程变化图

由于算法的局限性，在 Matlab 运行结果中满足每条切割路径都被切割的条件下，最优切割方案是
A1-A60-A61-A59-A58-A8-A6-A7-A9-A20-A21-A19-A18-A64-A62-A63-A65-A15-A17-A16-A14-A51-A50-A52-A53-A3-A2-A4-A5-A55-A54-A56-A57-A68-A66-A67-A69-A82-A83-A33-A31-A30-A32-A34-A36-A37-A35-A81-A80-A78-A79-A77-A76-A74-A75-A29-A28-A26-A27-A38-A40-A41-A39-A89-A87-A86-A88-A90-A91-A93-A92-A46-A47-A45-A43-A42-A44-A94-A96-A97-A95-A49-A48-A85-A84-A73-A71-A70-A72-A22-A23-A25-A24-A13-A11-A10-A12-A1，如图 13 所示，最短空程为 46.9680。

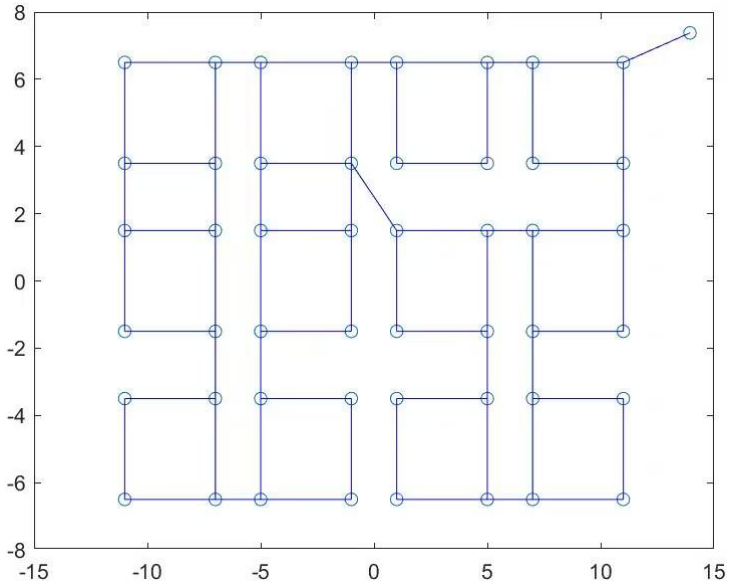


图 13 N3 局部切割路径最优方案图

但此时的最优解仅为椭圆内部 12 个小矩形的最短空程，还需加上第二问中切割外部的锯齿状切割线，4 个小圆和椭圆的最短空程，因此最终的切割路径如图 14 中红线所示，最短空程为 46.9680+31.5535=78.5215。

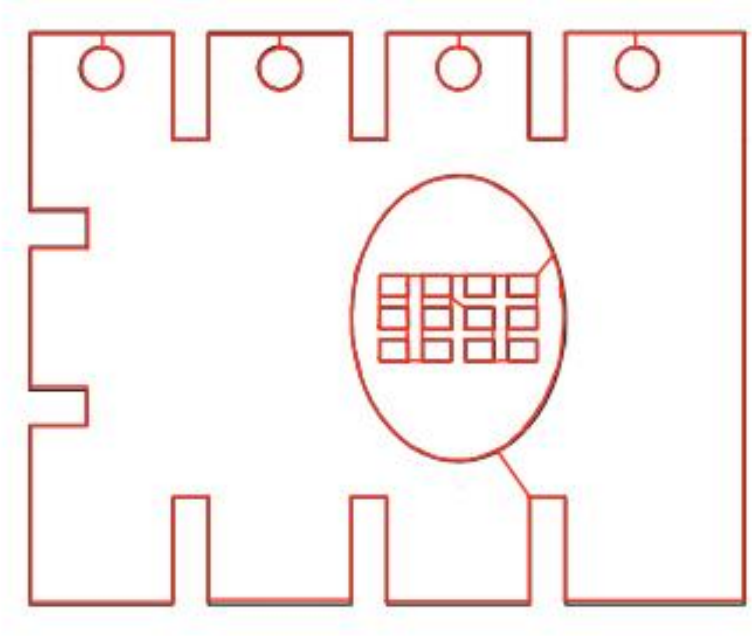


图 14 N3 实际切割路径图

5.4 问题四模型的建立与求解

5.4.1 绘制 N4 的下料切割示意图

1. 将图中所有拐点处标号。

2. 沿用问题一的思想，对组成需要折返路径的两个端点中位于锯齿状切割线上的那个点处新增一个点并标号。

3. 寻找内部小圆和椭圆上距离锯齿状切割线上最近的点并标号。

4. 为“过桥”的端点标号。

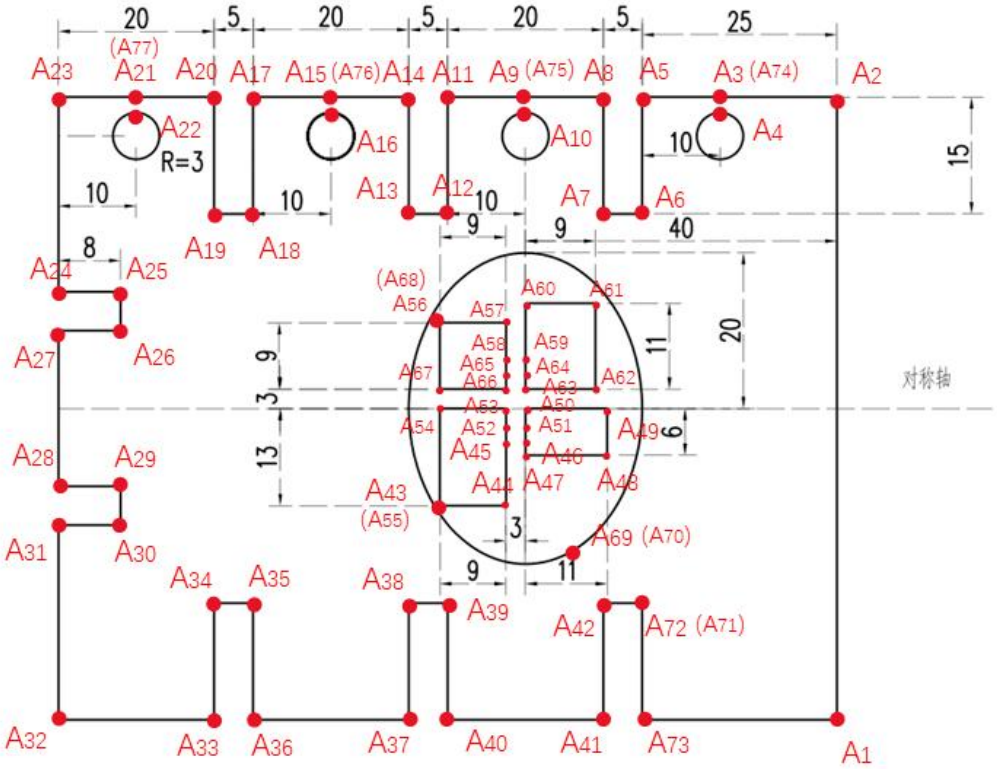


图 15 N4 下料切割示意图

5.4.2 切割方案的选取

经过计算发现，题目所给的需要添加过桥的嵌套矩形件套料图 N4 和图中所给的数据存在矛盾，点 A56 应在椭圆圆周上，点 A43 应在椭圆外，并非题目图中所画的均在椭圆内部，根据题中数据画出需要切割路径如图 16 中蓝线部分所示：

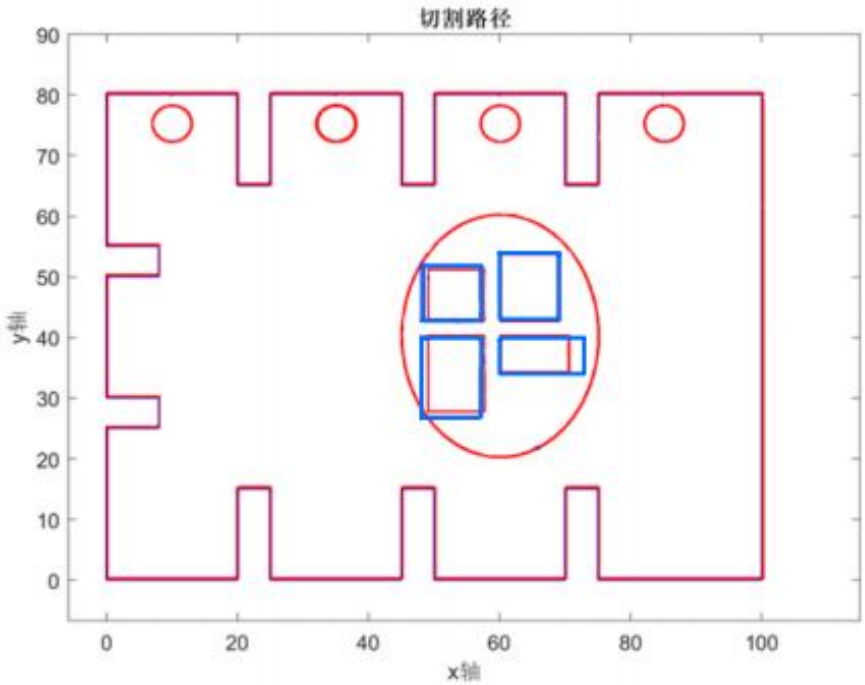


图 16 需要添加过桥的嵌套矩形件套料图 N4

方案一：当 A43 到椭圆的距离大于 3 时，如果从 A43 到椭圆的距离是空程的话就会加大空程，所以选择 A54 和 A67 间 3 的距离作为空程，此时“过桥”数为 2 或 3 是等价的。

方案二：A43 到椭圆的距离小于 3 时，就选择这条路径作为空程，此时会有 2 个“过桥”。

5.4.3 N4 最优切割路径模型的建立

（1）决策变量

切割路径的优化要明确的给出每一条路径是否需要经过，而每一条路径又是由两个点来确定的，所以最终的决策变量就是两点确定的路径是否需要经过。

$$x_{ij}=\begin{cases}1,\text{要经过}A_i\text{和}A_j\text{之间的路径}\\0,\text{否则}\end{cases}\quad(i,j=1,2,\cdots,77)\tag{1}$$

（2）目标函数

题目要求最优切割路径要满足空程最短的原则，即总的路径再减去有效切割的路径后要最短，得到的目标函数为：

$$\min\ z=\sum\sum x_{ij}\times d_{ij}(i,j=1,2,\cdots,77)-(54\Pi+746)\tag{2}$$

其中，d_{ij} 表示点 A_i 到点 A_j 之间的距离，$54\Pi+746$ 是有效切割长度之和。

（3）约束条件

1. 每个点被到达且仅被到达一次，每个点离开且只被离开一次。

2. $d_{ij}=\infty$(A_i与A_j表示同一位置的不同特征点)，即 A_i 和 A_j 点之间不会有切割路径。

3. $D_{43}<3$，即 A_{43} 到椭圆的最短距离小于 3。

5.4.4 N4 最优切割路径模型的求解

根据假设，此时 A43 到椭圆的距离是 0，小于 3，因此本问在计算时选择 A43 到椭圆的路径为空程，此时“过桥”数为 2。

对于不需要考虑过桥的封闭图形，如四个小圆，依据第二问取临近点表示该图形并对启停点进行路径优化。对于需要考虑过桥的矩形图形，设置 2 个与矩形小零件顶点的最短距离大于等于 1 的“过桥”，将过桥两端设置为启停点，并进行路径优化[4]。

点 A56、A43 在椭圆圆周上，本文利用 Matlab 软件由改进判断条件的模拟退火算法求解，最优切割方案是 A1-A73-A72-A71-A70-A68-A67-A66-A65-A64-A63-A62-A61-A60-A59-A58-A57-A56-A55-A54-A53-A52-A51-A50-A49-A48-A47-A46-A45-A44-A43-A69-A42-A41-A40-A39-A38-A37-A36-A35-A34-A33-A32-A31-A30-A29-A28-A27-A26-A25-A24-A23-A77-A22-A21-A20-A19-A18-A17-A76-A16-A15-A14-A13-A12-A11-A75-A10-A9-A8-A7-A6-A5-A74-A4-A3-A2-A1，如图 17 所示，最短空程为 43.5535。

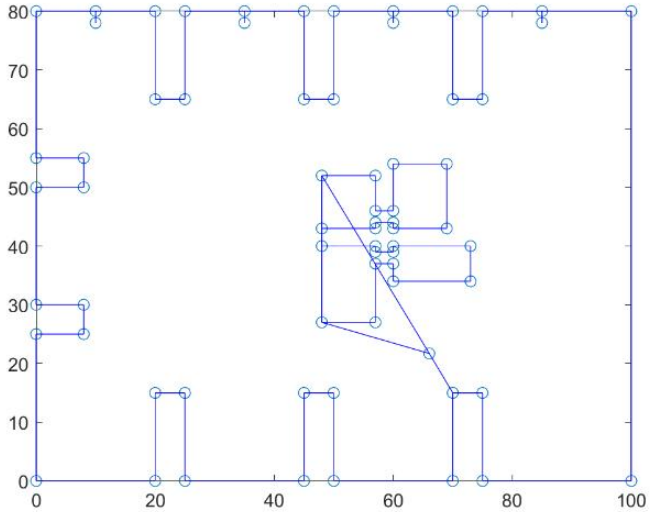


图 17 N2 切割路径最优方案图

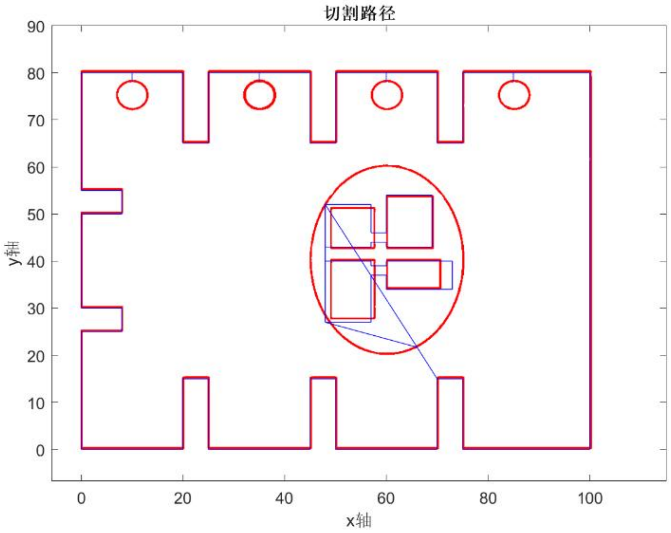


图 18 N3 实际切割路径图

六、 模型的评价

6.1 模型的优点

1.TSP 问题要求每个点只能经过一次，但是在实际计算时发现如果仅经过一次有的切割路径就会确实，因此为了不违背算法又解决切割路径缺失的问题，巧妙地把同一个位置标记两次，且规定标记的两个点之间的距离是无穷，这样两个点之间不仅不会有切割路径而且还做到了重合。

2.在设计切割方案时没有一味地追求最优切割方案，最短空程距离，同时考虑到了实际操作的难度，将每一个内孔零件看做一个闭合回路，每次切割都是一次性沿着闭合回路完成该零件的切割。

6.2 模型的缺点

1.本文所求最优方案和最优值不一定是全局最优解，但却是十分接近最优解的近似解。

2．跳出局部最优解缓慢：模拟退火算法的核心在于其接受劣解的能力，这有助于跳出局部最优解。然而，当陷入较差的局部最优时，算法跳出这一状态的速度可能较慢，得出全局最优解的迭代次数可能更长。

七、 参考文献

[1]姜启源等编，数学建模案例选集[M](第 1 版)，北京，高等教育出版社，2006.1.
[2]李泳,张宝峰.复杂轮廓激光切割路径优化算法的研究[J].天津理工大学学报,2007(03):76-79.
[3]熊慧,黄菊永.基于贪心启发式算法的多目标二维切割问题[J].电子技术与软件工程,2016(24):154-155.
[4]张文斌,廖小平,马俊燕,等.考虑桥连接的钣金切割件加工路径优化方法[J].锻压技术,2023,48(04):66-71.DOI:10.13330/j.issn.1000-3940.2023.04.009.

附录

附录 1

介绍：该代码是 Matlab 编写的，作用是求解 N1 最优切割路径（修改前）

```
%% 清除一切障碍
clc;
clear;
%% 模拟退火算法——钢材切割问题
x_y = [60,15;60,35;20,35;20,15;80,0;80,50;0,50;0,0]; % 输入笛卡尔坐标系位置
n = size(x_y,1);  % 点的数目
figure  % 新建 1 个图形窗口
plot(x_y(:,1),x_y(:,2),'o');   % 画出每个特征点的分布散点图(特征点即顶点和拐点)
hold on
d = zeros(n);   % 初始化所有城市的距离矩阵 d 全为 0
for i = 2:n  % 外循环 i 从 2 开始是因为城市 1 与城市 1 距离为 0，而后算下三角矩阵再对称即可得到距离矩阵
    for j = 1:i  % 内循环 j 得到每个城市之间的坐标和距离
        x_y_i = x_y(i,:);   x_i = x_y_i(1);     y_i = x_y_i(2);  % 城市 i 的横坐标为 x_i 而纵坐标为 y_i
        x_y_j = x_y(j,:);   x_j = x_y_j(1);     y_j = x_y_j(2);  % 城市 j 的横坐标为 x_j 而纵坐标为 y_j
        d(i,j) = sqrt((x_i-x_j)^2 + (y_i-y_j)^2);   % 算出城市 i 与 j 的距离
    end
end
% 切割线部分的路径值设为 0(必须放在下三角矩阵中，即 i>j)
d(2,1)=0;
d(4,1)=0;
d(3,2)=0;
d(4,3)=0;
d(6,5)=0;
d(7,6)=0;
d(8,5)=0;
d = d+d';   % 生成距离矩阵的上三角矩阵
%% 参数初始化
T0 = 1000;   % 模拟退火模型初始温度
T = T0; % 随着迭代变化，温度也会发生改变，第一次迭代时温度为 T0
maxgen = 1000;  % 设置最大迭代次数
Lk = 500;  % 变化后不同温度下的迭代次数
beta = 0.95;  % 设置温度衰减系数
%%  随机生成初始解
path0 = randperm(n);  % 随机生成一个 1-n 的序列作为初始的路径 path0，随后不断改进
best_path=path0; % 默认初始路径为最优路径
```

```
result0 = jlh(path0,d); % 调用 jlh 函数计算当前路径的距离，随后不断改进
%% 定义中间变量，便于输出结果和绘图
min_result = result0;     % 初始化路径下对应的距离为 result0
RESULT = zeros(maxgen,1); % 每一次外层循环结束后找到的 min_result 然后绘图观察某段时间后值最小且不变的情况即为所求优化解
%% 模拟退火过程
for iter = 1 : maxgen  % 外循环 iter，采用指定最大迭代次数
    for i = 1 : Lk  %  内循环，即在每个温度下开始迭代，整体共迭代 maxgen*Lk 次
        path1 = new_path(path0);  % 调用 new_path 函数生成新的路径
        result1 = jlh(path1,d); % 计算新路径下的距离
        % 条件判断 若新解距离短，则把新解的值赋值给原来的解
        if result1 < result0
            path0 = path1; % 更新当前路径为新路径
            result0 = result1; % 更新最短路径的和值
        else
            p = exp(-(result1 - result0)/T); % 根据 Metropolis 准则计算一个可接受概率
            if rand(1) < p   % 生成一个随机数和这个概率比较，如果该随机数小于这个概率则接受新解
                path0 = path1;  % 更新当前路径为新路径
                result0 = result1; % 更新最短的路径的和值
            end
        end
        % 判断是否要更新找到的最佳的解值
        if result0 < min_result  % 比较最短路径的和值，若当前解更好，则对其进行更新
            min_result = result0;  % 更新最小的距离数值
            best_path = path0;  % 更新为找到的最短路径
        end
    end
    RESULT(iter) = min_result; % 保存本轮外循环结束后找到的最小距离在 RESULT 里方便观察
    T = beta*T;   % 温度下降法则
end
% 按顺序和约束起始序号排序
targetValue=5;
index = find(best_path == targetValue, 1, 'first'); % 找序号为 5 的作为切割点
a=best_path(1,1:index-1);
b=best_path(index:end);
best_path=[b,a];
best_path = [best_path,best_path(1)];   % 在最短路径的最后面加入第一个点的元素(闭合的)
```

```
disp('最佳的方案是：'); disp(mat2str(best_path)) % 显示最佳路径(闭合的)
disp('此时最优值是：'); disp(min_result) % 显示最短路径的和值
n = n+1;  % 点的个数加一个(因为加了个闭合的重复的点)
for i = 1:n-1
    j = i+1;
    x_y_i = x_y(best_path(i),:);   x_i = x_y_i(1);     y_i = x_y_i(2); % x_i 为点坐标 x 轴位置 y_i 为点坐标 y 轴位置
    x_y_j = x_y(best_path(j),:);   x_j = x_y_j(1);     y_j = x_y_j(2); % 此为最优路线下后一个城市的坐标
    plot([x_i,x_j],[y_i,y_j],'-b')    % 不断画图，每两个点就作出一条线段，直到所有的点
    pause(0.5)  % 缓冲 0.5s 再画下一条线段
    hold on % 准备继续画图
end
%% 画出最短路径随迭代次数增加而变化的图形
figure
plot(1:maxgen,RESULT,'b-'); % 自变量为迭代次数 1:maxgen，因变量 RESULT
xlabel('迭代次数'); % x 轴名称
    ylabel('最短路径的和值'); % y 轴名称
function  result =  jlh(path,d) % 定义函数算距离和
% 输入：path:旧路径（1 至 n 的一个序列），d：生成距离矩阵
    n = length(path);
    result = 0; % 初始化该路径走的距离为 0
    for i = 1:n-1
        result = d(path(i),path(i+1)) + result;  % 需要按照这个序列更新走过的路程这个值
    end
    result = d(path(1),path(n)) + result;  % 加上从最后一个点返回到最开始那个点的距离
    end
function path1 = new_path(path0) % path0: 代入原来的路径
    n = length(path0); % n 为点数量
    % 随机选择两种生成新路径的方法
    p1 = 0.33;  % 自行设置使用交换法产生新路径的概率
    p2 = 0.33;  % 自行设置使用移位法产生新路径的概率
    r = rand(1); % 生成一个[0 1]间均匀分布的随机数
    % 方法一
    if  r< p1    % 用交换法产生新路径
        c1 = randi(n);   % 产生 1-n 中的一个随机整数
        c2 = randi(n);   % 产生 1-n 中的一个随机整数
        path1 = path0;  % 将 path0 的值给 path1
        path1(c1) = path0(c2);  %改 path1 第 c1 个位置的元素为 path0 第 c2 个位置的元素
```

```
        path1(c2) = path0(c1);  %改 path1 第 c2 个位置的元素为 path0 第 c1 个位置的元素
    % 方法二
    elseif r < p1+p2 % 使用移位法产生新路径
        c1 = randi(n);   % 产生 1-n 中的一个随机整数
        c2 = randi(n);   % 产生 1-n 中的一个随机整数
        c3 = randi(n);   % 产生 1-n 中的一个随机整数
        sort_c = sort([c1 c2 c3]);  % 对 c1 c2 c3 从小到大进行排序
        c1 = sort_c(1);  c2 = sort_c(2);  c3 = sort_c(3);  % c1 < = c2 <=  c3
        tem1 = path0(1:c1-1);
        tem2 = path0(c1:c2);
        tem3 = path0(c2+1:c3);
        tem4 = path0(c3+1:end);
        path1 = [tem1 tem3 tem2 tem4];
    % 方法三
    else  % 使用倒置法产生新路径
        c1 = randi(n);   % 产生 1-n 中的一个随机整数
        c2 = randi(n);   % 产生 1-n 中的一个随机整数
        if c1>c2  % 若 c1 比 c2 大，就交换 c1 和 c2 的值
            tem = c2;
            c2 = c1;
            c1 = tem;
        end
        tem1 = path0(1:c1-1);
        tem2 = path0(c1:c2);
        tem3 = path0(c2+1:end);
        path1 = [tem1 fliplr(tem2) tem3];   %矩阵的左右对称翻转 fliplr，上下对称翻转 flipud
    end
    end
```

附录 2

介绍：该代码是 Matlab 编写的，作用是求解 N1 最优切割路径（修改后）

```
%% 清除一切障碍
clc;
clear;
%% 模拟退火算法—钢材切割问题
x_y = [60,15;60,35;20,35;20,15;80,0;80,50;0,50;0,0;20,35]; % 输入笛卡尔坐标系位置
n = size(x_y,1);  % 点的数目
figure  % 新建 1 个图形窗口
plot(x_y(:,1),x_y(:,2),'o');   % 画出每个特征点的分布散点图(特征点即顶点和拐点)
hold on
d = zeros(n);   % 初始化所有城市的距离矩阵 d 全为 0
```

```
for i = 2:n  % 外循环 i 从 2 开始是因为城市 1 与城市 1 距离为 0，而后算下三角矩阵再对称即可得到距离矩阵
    for j = 1:i  % 内循环 j 得到每个城市之间的坐标和距离
        x_y_i = x_y(i,:);   x_i = x_y_i(1);     y_i = x_y_i(2);  % 城市 i 的横坐标为 x_i 而纵坐标为 y_i
        x_y_j = x_y(j,:);   x_j = x_y_j(1);     y_j = x_y_j(2);  % 城市 j 的横坐标为 x_j 而纵坐标为 y_j
        d(i,j) = sqrt((x_i-x_j)^2 + (y_i-y_j)^2);   % 算出城市 i 与 j 的距离
    end
end
% 切割线部分的路径值设为 0(必须放在下三角矩阵中，即 i>j)
d(2,1)=0;
d(4,1)=0;
d(3,2)=0;
d(4,3)=0;
d(6,5)=0;
d(7,6)=0;
d(8,5)=0;
d(9,3)=inf; % 防止自身相连
d(9,2)=0;
d(9,4)=0;
d = d+d';   % 生成距离矩阵的上三角矩阵
%% 参数初始化
T0 = 1000;   % 模拟退火模型初始温度
T = T0; % 随着迭代变化，温度也会发生改变，第一次迭代时温度为 T0
maxgen = 1000;  % 设置最大迭代次数
Lk = 500;  % 变化后不同温度下的迭代次数
beta = 0.95;  % 设置温度衰减系数
%%  随机生成初始解
path0 = randperm(n);  % 随机生产一个 1-n 的序列作为初始的路径 path0，随后不断改进
best_path=path0; % 默认初始路径为最优路径
result0 = jlh(path0,d); % 调用 jlh 函数计算当前路径的距离，随后不断改进
%% 定义中间变量，便于输出结果和绘图
min_result = result0;     % 初始化路径下对应的距离为 result0
RESULT = zeros(maxgen,1); % 每一次外层循环结束后找到的 min_result 然后绘图观察某段时间后值最小且不变的情况即为所求优化解
%% 模拟退火过程
for iter = 1 : maxgen  % 外循环 iter，采用指定最大迭代次数
    for i = 1 : Lk  %  内循环，即在每个温度下开始迭代，整体共迭代 maxgen*Lk 次
        path1 = new_path(path0);  % 调用 new_path 函数生成新的路径
        result1 = jlh(path1,d); % 计算新路径下的距离
        % 条件判断 若新解距离短，则把新解的值赋值给原来的解
```

```
        if result1 < result0
            path0 = path1; % 更新当前路径为新路径
            result0 = result1; % 更新最短路径的和值
        else
            p = exp(-(result1 - result0)/T); % 根据 Metropolis
准则计算一个可接受概率
            if rand(1) < p   % 生成一个随机数和这个概率比较，如果该随
机数小于这个概率则接受新解
                path0 = path1;  % 更新当前路径为新路径
                result0 = result1; % 更新最短的路径的和值
            end
        end
        % 判断是否要更新找到的最佳的解值
        if result0 < min_result  % 比较最短路径的和值，若当前解更好，
则对其进行更新
            min_result = result0;  % 更新最小的距离数值
            best_path = path0;  % 更新为找到的最短路径
        end
    end
    RESULT(iter) = min_result; % 保存本轮外循环结束后找到的最小距离
在 RESULT 里方便观察
    T = beta*T;   % 温度下降法则
end
% 按顺序和约束起始序号排序
targetValue=5;
index = find(best_path == targetValue, 1, 'first'); % 找序号为
5 的作为切割点
a=best_path(1,1:index-1);
b=best_path(index:end);
best_path=[b,a];
best_path = [best_path,best_path(1)];   % 在最短路径的最后面加入
第一个点的元素(闭合的)
disp('最佳的方案是：');disp(mat2str(best_path)) % 显示最佳路径(闭
合的)
disp('此时最优值是：'); disp(min_result) % 显示最短路径的和值
n = n+1;  % 点的个数加一个(因为加了个闭合的重复的点)
for i = 1:n-1
    j = i+1;
    x_y_i = x_y(best_path(i),:);   x_i = x_y_i(1);     y_i =
x_y_i(2); % x_i 为点坐标 x 轴位置 y_i 为点坐标 y 轴位置
    x_y_j = x_y(best_path(j),:);   x_j = x_y_j(1);     y_j =
x_y_j(2); % 此为最优路线下后一个城市的坐标
    plot([x_i,x_j],[y_i,y_j],'-b')    % 不断画图，每两个点就作出一
条线段，直到所有的点
    pause(0.5)  % 缓冲 0.5s 再画下一条线段
    hold on % 准备继续画图
```

```
end
%% 画出最短路径随迭代次数增加而变化的图形
figure
plot(1:maxgen,RESULT,'b-'); % 自变量为迭代次数 1:maxgen，因变量
RESULT
xlabel('迭代次数'); % x 轴名称
    ylabel('最短路径的和值'); % y 轴名称
```

附录 3

介绍：该代码是 Matlab 编写的，作用是求解 N2 最优切割路径

```
clc;
clear;
%% 模拟退火算法—钢材切割问题
% 根据 tuoyuan.m 文件求解到椭圆最短距离点得到圆周上的点 x_m 和 y_m
x_m = 66.0560825404235;
y_m = 21.7025143554360;
x_y = [100,0; 100,80; 85,80; 85,78; 75,80; 75,65; 70,65; 70,80;
60,80; 60,78; 50,80; 50,65; 45,65; 45,80; 35,80; 35,78; 25,80;
25,65; 20,65; 20,80; 10,80; 10,78; 0,80; 0,55; 8,55; 8,50; 0,50;
0,30; 8,30; 8,25;0,25; 0,0; 20,0; 20,15; 25,15; 25,0; 45,0; 45,15;
50,15; 50,0; 70,0; 70,15; x_m,y_m; 75,15; 75,0; 85,80; 60,80;
35,80; 10,80; 70,15]; % 输入笛卡尔坐标系位置
n = size(x_y,1);  % 点的数目
figure  % 新建 1 个图形窗口
% 画出每个特征点的分布散点图(特征点即顶点和拐点)
plot(x_y(:,1),x_y(:,2),'o');
hold on
d = zeros(n);   % 初始化所有城市的距离矩阵 d 全为 0
for i = 2:n  % 外循环 i 从 2 开始是因为城市 1 与城市 1 距离为 0，而后算下三
角矩阵再对称即可得到距离矩阵
    for j = 1:i  % 内循环 j 得到每个城市之间的坐标和距离
        x_y_i = x_y(i,:);   x_i = x_y_i(1);     y_i = x_y_i(2);  %
城市 i 的横坐标为 x_i 而纵坐标为 y_i
        x_y_j = x_y(j,:);   x_j = x_y_j(1);     y_j = x_y_j(2);  %
城市 j 的横坐标为 x_j 而纵坐标为 y_j
        d(i,j) = sqrt((x_i-x_j)^2 + (y_i-y_j)^2);   % 算出城市 i
与 j 的距离
    end
end
% 切割线部分的路径值设为 0(必须放在下三角矩阵中，即 i>j)
d(2,1)=0;d(3,2)=0;d(4,3)=2;d(46,4)=2;d(46,5)=0;d(6,5)=0;d(7,6
)=0;d(8,7)=0;d(9,8)=0;d(10,9)=2;d(47,10)=2;d(47,11)=0;
d(12,11)=0;d(13,12)=0;d(14,13)=0;d(15,14)=0;d(16,15)=2;d(48,1
6)=2;d(48,17)=0;d(18,17)=0;d(19,18)=0;d(20,19)=0;
d(21,20)=0;d(22,21)=2;d(49,22)=2;d(49,23)=0;d(24,23)=0;d(25,2
4)=0;d(26,25)=0;d(27,26)=0;d(28,27)=0;d(29,28)=0;
d(30,29)=0;d(31,30)=0;d(32,31)=0;d(33,32)=0;d(34,33)=0;d(35,3
```

```
4)=0;d(36,35)=0;d(37,36)=0;d(38,37)=0;d(39,38)=0;
d(40,39)=0;d(41,40)=0;d(42,41)=0;d(43,42)=7.776772056115416;d
(50,43)=7.77677205611541;d(50,44)=0;d(45,44)=0;d(45,1)=0;
% 防止自身相连
d(46,3)=1000;
d(47,9)=1000;
d(48,15)=1000;
d(49,21)=1000;
d(50,42)=1000;
% 生成距离矩阵的上三角矩阵
d = d+d';
%% 参数初始化
T0 = 1000;   % 模拟退火模型初始温度
T = T0; % 随着迭代变化，温度也会发生改变，第一次迭代时温度为 T0
maxgen = 1000;  % 设置最大迭代次数
Lk = 500;  % 变化后不同温度下的迭代次数
beta = 0.95;  % 设置温度衰减系数
%%  随机生成初始解
path0 = randperm(n);  % 随机生成一个 1-n 的序列作为初始的路径 path0，
随后不断改进
best_path=path0; % 默认初始路径为最优路径
result0 = jlh(path0,d); % 调用 jlh 函数计算当前路径的距离，随后不断改
进
%% 定义中间变量，便于输出结果和绘图
min_result = result0;     % 初始化路径下对应的距离为 result0
RESULT = zeros(maxgen,1); % 每一次外层循环结束后找到的 min_result 然
后绘图观察某段时间后值最小且不变的情况即为所求优化解
%% 模拟退火过程
for iter = 1 : maxgen  % 外循环 iter，采用指定最大迭代次数
    for i = 1 : Lk  %  内循环,即在每个温度下开始迭代,整体共迭代 maxgen*Lk
次
        path1 = new_path(path0);  % 调用 new_path 函数生成新的路径
        result1 = jlh(path1,d); % 计算新路径下的距离
        % 条件判断 若新解距离短，则把新解的值赋值给原来的解
        if result1 < result0
            path0 = path1; % 更新当前路径为新路径
            result0 = result1; % 更新最短路径的和值
        else
            p = exp(-(result1 - result0)/T); % 根据 Metropolis 准
则计算一个可接受概率
            if rand(1) < p   % 生成一个随机数和这个概率比较，如果该随机
数小于这个概率则接受新解
                path0 = path1;  % 更新当前路径为新路径
                result0 = result1; % 更新最短的路径的和值
            end
        end
```

```
        % 判断是否要更新找到的最佳的解值
        if result0 < min_result  % 比较最短路径的和值，若当前解更好，
则对其进行更新
            min_result = result0;  % 更新最小的距离数值
            best_path = path0;  % 更新为找到的最短路径
        end
    end
    RESULT(iter) = min_result; % 保存本轮外循环结束后找到的最小距离在
RESULT 里方便观察
    T = beta*T;   % 温度下降法则
end
% 按顺序和约束起始序号排序
targetValue=1;
index = find(best_path == targetValue, 1, 'first'); % 找序号作为
切割点
a=best_path(1,1:index-1);
b=best_path(index:end);
best_path=[b,a];
best_path = [best_path,best_path(1)];   % 在最短路径的最后面加入第
一个点的元素(闭合的)
disp('最佳的方案是: '); disp(mat2str(best_path)) % 显示最佳路径(闭
合的)
disp('此时最优值是: '); disp(min_result) % 显示最短路径的和值
n = n+1;  % 点的个数加一个(因为加了个闭合的重复的点)
for i = 1:n-1
    j = i+1;
    x_y_i = x_y(best_path(i),:);   x_i = x_y_i(1);     y_i =
x_y_i(2); % x_i 为点坐标 x 轴位置 y_i 为点坐标 y 轴位置
    x_y_j = x_y(best_path(j),:);   x_j = x_y_j(1);     y_j =
x_y_j(2); % 此为最优路线下后一个城市的坐标
    plot([x_i,x_j],[y_i,y_j],'-b')    % 不断画图，每两个点就作出一条
线段，直到所有的点
    pause(0.02)  % 缓冲 0.5s 再画下一条线段
    hold on % 准备继续画图
end
%% 画出最短路径随迭代次数增加而变化的图形
figure
plot(1:maxgen,RESULT,'b-'); % 自变量为迭代次数 1:maxgen，因变量
RESULT
xlabel('迭代次数'); % x 轴名称
ylabel('最短路径的和值'); % y 轴名称
function  result =  jlh(path,d) % 定义函数算距离和
% 输入：path:旧路径（1 至 n 的一个序列），d：生成距离矩阵
    n = length(path);
    result = 0; % 初始化该路径走的距离为 0
    for i = 1:n-1
```

```
        result = d(path(i),path(i+1)) + result;  % 需要按照这个序列更新走过的路程这个值
    end
    result = d(path(1),path(n)) + result;  % 加上从最后一个点返回到最开始那个点的距离
end
function path1 = new_path(path0) % path0: 代入原来的路径
    n = length(path0); % n 为点数量
    % 随机选择两种生成新路径的方法
    p1 = 0.33;  % 自行设置使用交换法产生新路径的概率
    p2 = 0.33;  % 自行设置使用移位法产生新路径的概率
    r = rand(1); % 生成一个[0 1]间均匀分布的随机数
    % 方法一
    if  r< p1    % 用交换法产生新路径
        c1 = randi(n);   % 产生 1-n 中的一个随机整数
        c2 = randi(n);   % 产生 1-n 中的一个随机整数
        path1 = path0;  % 将 path0 的值给 path1
        path1(c1) = path0(c2);  %改 path1 第 c1 个位置的元素为 path0 第 c2 个位置的元素
        path1(c2) = path0(c1);  %改 path1 第 c2 个位置的元素为 path0 第 c1 个位置的元素
    % 方法二
    elseif r < p1+p2 % 使用移位法产生新路径
        c1 = randi(n);   % 产生 1-n 中的一个随机整数
        c2 = randi(n);   % 产生 1-n 中的一个随机整数
        c3 = randi(n);   % 产生 1-n 中的一个随机整数
        sort_c = sort([c1 c2 c3]);  % 对 c1 c2 c3 从小到大进行排序
        c1 = sort_c(1);  c2 = sort_c(2);  c3 = sort_c(3);  % c1 < = c2 <=  c3
        tem1 = path0(1:c1-1);
        tem2 = path0(c1:c2);
        tem3 = path0(c2+1:c3);
        tem4 = path0(c3+1:end);
        path1 = [tem1 tem3 tem2 tem4];
    % 方法三
    else  % 使用倒置法产生新路径
        c1 = randi(n);   % 产生 1-n 中的一个随机整数
        c2 = randi(n);   % 产生 1-n 中的一个随机整数
        if c1>c2  % 若 c1 比 c2 大，就交换 c1 和 c2 的值
            tem = c2;
            c2 = c1;
            c1 = tem;
        end
        tem1 = path0(1:c1-1);
        tem2 = path0(c1:c2);
```

```
        tem3 = path0(c2+1:end);
        path1 = [tem1 fliplr(tem2) tem3];   %矩阵的左右对称翻转 fliplr，上下对称翻转 flipud
    end
end
```

附录 4

介绍：该代码是 Matlab 编写的，作用是寻找椭圆上与锯齿形切割线距离最近的点

```
close all; clear all; clc
%% input: k x0 y0 xc yc a b
%% output: ans_x ans_y ans_dis
% [ans_x ans_y]为椭圆上距离点[x0 y0]最近的点，ans_dis 为相应的距离
%% 以点到椭圆中心直线 与 椭圆 的交点为起点，进行搜索，可缩短搜寻范围
k = 0.0000001; %设置精度
x0 = 70; y0 = 15;% [x0,y0] 为点
% 椭圆为：(x-xc).^2/a.^2+(y-yc).^2/b.^2==1
xc = 60; yc = 40; % [xc,yc] 为椭圆中心
a = 15; b = 20;
x1 = xc - a; x2 = xc + a;
y1 = yc - b; y2 = yc + b;
fourpoint = [x1 yc;x2 yc;xc y1;xc y2];% 四个端点
syms x y
start =
solve((x-xc).^2/a.^2+(y-yc).^2/b.^2==1,y-y0==((yc-y0)/(xc-x0))*(x-x0),x,y
);
node_x = double(start.x);
node_y = double(start.y);
[~,index] = min(abs(node_x - x0));
start_xy = [node_x(index),node_y(index)];
[~,index] = min(sum((fourpoint - [x0 y0]).^2,2));
find_x = min(start_xy(1),fourpoint(index,1));
end_x = max(start_xy(1),fourpoint(index,1));
err = k;
while err >= k
    m = 5;
    x_sam = [find_x:(end_x-find_x)/m:end_x];
    y_sam = [];
    for i = 1:(m+1)
        f = sym((x-xc).^2/a.^2+(y-yc).^2/b.^2-1);
        f = subs(f,x,x_sam(i));
        y_sam(:,i) = eval(solve(f,y));
    end
    [~,index] = min(abs(y_sam(:,1)-start_xy(2)));
    y_sam = y_sam(index,:);
```

```
    dis_sam = sum(([x_sam',y_sam'] - [x0 y0]).^2,2);
    [~,index] = min(dis_sam);
    if index <= m & index > 1;
        err = abs(dis_sam(index-1)-dis_sam(index+1));
        find_x = x_sam(index-1); end_x = x_sam(index+1);
        ans_x = x_sam(index); ans_y = y_sam(index);
        ans_dis = sqrt(dis_sam(index));
    elseif index == 1;
        err = abs(dis_sam(index)-dis_sam(index+1));
        find_x = x_sam(index); end_x = x_sam(index+1);
        ans_x = x_sam(index); ans_y = y_sam(index);
        ans_dis = sqrt(dis_sam(index));
    else index == m+1;
        err = abs(dis_sam(index-1)-dis_sam(index));
        find_x = x_sam(index-1); end_x = x_sam(index);
        ans_x = x_sam(index); ans_y = y_sam(index);
        ans_dis = sqrt(dis_sam(index));
    end
end
%% 画图
t = [0:0.1:2*pi];
x = xc + a*cos(t);
y = yc + b*sin(t);
plot(x,y) ; hold on
plot([xc x0],[yc y0]); hold on
plot([ans_x,x0],[ans_y,y0]); hold on
scatter(start_xy(1),start_xy(2),'r*'); hold on
scatter(ans_x,ans_y,'r*')
```

附录 5

介绍：该代码是 Matlab 编写的，作用是求解 N3 最优切割路径

```
%% 清除一切障碍
clc;
clear;
%% 模拟退火算法—钢材切割问题
% 椭圆内特征切点位置
x=13.94;
y=7.38;
% 批量赋值点的位置
for i = 1:4
    x_index=11-6*(i-1);

y_add=[-6.5;-6.5;-3.5;-3.5;-1.5;-1.5;1.5;1.5;3.5;3.5;6.5;6.5]
;
    y=[y;y_add];
```

```
    for j = 1:6
        x=[x;x_index];
        x=[x;x_index-4];
    end
end
x_y=[];
% 点位置放入 x_y 中
for i = 1:length(x)
    mid=[x(i),y(i)];
    x_y = [x_y;mid]; % 输入
    % 笛卡尔坐标系位置
end
x_y_1=x_y(2:end,:);
x_y=[x_y;x_y_1];
%
x_y=[x_y;11,-3.5;11,1.5;11,6.5;5,-3.5;5,1.5;5,6.5;-1,-3.5;-1,
1.5;-1,6.5;-7,-3.5;-7,1.5;-7,6.5]; % 插入折返点
n = size(x_y,1);  % 点的数目
figure  % 新建 1 个图形窗口
plot(x_y(:,1),x_y(:,2),'o');   % 画出每个特征点的分布散点图(特征点
即顶点和拐点)
hold on
d = zeros(n);   % 初始化所有城市的距离矩阵 d 全为 0
for i = 2:n  % 外循环 i 从 2 开始是因为城市 1 与城市 1 距离为 0，而后算下三
角矩阵再对称即可得到距离矩阵
    for j = 1:i  % 内循环 j 得到每个城市之间的坐标和距离
        x_y_i = x_y(i,:);   x_i = x_y_i(1);     y_i = x_y_i(2);  %
城市 i 的横坐标为 x_i 而纵坐标为 y_i
        x_y_j = x_y(j,:);   x_j = x_y_j(1);     y_j = x_y_j(2);  %
城市 j 的横坐标为 x_j 而纵坐标为 y_j
        d(i,j) = sqrt((x_i-x_j)^2 + (y_i-y_j)^2);   % 算出城市 i
与 j 的距离
    end
end
% 切割路线空程为 0
for i = 1:24
    i_0=4*i-2;
    i_1=i_0+1;
    i_2=i_1+1;
    i_3=i_2+1;
    d(i_3,i_2)=0;
    d(i_3,i_1)=0;
    d(i_2,i_0)=0;
    d(i_1,i_0)=0;
end
% 相同点之间间距为无穷,周围的切割路径为 0
for i = 2:49
    i_0=i;
```

```
    iq_0=i_0+48;
    d(iq_0,i_0)=inf;
end
% 切割线部分的路径值设为 0(必须放在下三角矩阵中，即 i>j)
d = d+d';   % 生成距离矩阵的上三角矩阵
%% 参数初始化
T0 = 1000;   % 模拟退火模型初始温度
T = T0; % 随着迭代变化，温度也会发生改变，第一次迭代时温度为 T0
maxgen = 1000;  % 设置最大迭代次数
Lk = 500;  % 变化后不同温度下的迭代次数
beta = 0.95;  % 设置温度衰减系数
%%  随机生成初始解
rng(67);
path0 = randperm(n);  % 随机生产一个 1-n 的序列作为初始的路径 path0，随后不断改进
best_path=path0; % 默认初始路径为最优路径
result0 = jlh(path0,d); % 调用 jlh 函数计算当前路径的距离，随后不断改进
%% 定义中间变量，便于输出结果和绘图
min_result = result0;     % 初始化路径下对应的距离为 result0
RESULT = zeros(maxgen,1); % 每一次外层循环结束后找到的 min_result 然后绘图观察某段时间后值最小且不变的情况即为所求优化解
%% 模拟退火过程
for iter = 1 : maxgen  % 外循环 iter，采用指定最大迭代次数
    for i = 1 : Lk  %  内循环,即在每个温度下开始迭代,整体共迭代 maxgen*Lk 次
        path1 = new_path(path0);  % 调用 new_path 函数生成新的路径
        result1 = jlh(path1,d); % 计算新路径下的距离
        % 条件判断 若新解距离短，则把新解的值赋值给原来的解
        if result1 < result0
            path0 = path1; % 更新当前路径为新路径
            result0 = result1; % 更新最短路径的和值
        else
            p = exp(-(result1 - result0)/T); % 根据 Metropolis 准则计算一个可接受概率
            if rand(1) < p   % 生成一个随机数和这个概率比较，如果该随机数小于这个概率则接受新解
                path0 = path1;  % 更新当前路径为新路径
                result0 = result1; % 更新最短的路径的和值
            end
        end
        % 判断是否要更新找到的最佳的解值
        if result0 < min_result  % 比较最短路径的和值，若当前解更好，则对其进行更新
            min_result = result0;  % 更新最小的距离数值
```

```
            best_path = path0;  % 更新为找到的最短路径
        end
    end
    RESULT(iter) = min_result; % 保存本轮外循环结束后找到的最小距离在RESULT 里方便观察
    T = beta*T;   % 温度下降法则
end
% 按顺序排列，第一个元素为出发点
targetValue=1;
index = find(best_path == targetValue, 1, 'first'); % 找对应序号的作为切割点
a=best_path(1,1:index-1);
b=best_path(index:end);
best_path=[b,a];
best_path = [best_path,best_path(1)];   % 在最短路径的最后面加入第一个点的元素(闭合的)
disp('最佳的方案是: '); disp(mat2str(best_path)) % 显示最佳路径(闭合的)
disp('此时最优值是: '); disp(min_result) % 显示最短路径的和值
n = n+1;  % 点的个数加一个(因为加了个闭合的重复的点)
for i = 1:n-1
    j = i+1;
    x_y_i = x_y(best_path(i),:);   x_i = x_y_i(1);     y_i = x_y_i(2); % x_i 为点坐标 x 轴位置 y_i 为点坐标 y 轴位置
    x_y_j = x_y(best_path(j),:);   x_j = x_y_j(1);     y_j = x_y_j(2); % 此为最优路线下后一个城市的坐标
    plot([x_i,x_j],[y_i,y_j],'-b')    % 不断画图，每两个点就作出一条线段，直到所有的点
    pause(0.02)  % 缓冲 0.02s 再画下一条线段
    hold on % 准备继续画图
end
%% 画出最短路径随迭代次数增加而变化的图形
figure
plot(1:maxgen,RESULT,'b-'); % 自变量为迭代次数 1:maxgen，因变量RESULT
xlabel('迭代次数'); % x 轴名称
ylabel('最短路径的和值'); % y 轴名称
function path1 = new_path(path0) % path0: 代入原来的路径
    n = length(path0); % n 为点数量
    % 随机选择两种生成新路径的方法
    p1 = 0.33;  % 自行设置使用交换法产生新路径的概率
    p2 = 0.33;  % 自行设置使用移位法产生新路径的概率
    r = rand(1); % 生成一个[0 1]间均匀分布的随机数
    % 方法一
    if  r< p1    % 用交换法产生新路径
        c1 = randi(n);   % 产生 1-n 中的一个随机整数
```

```
        c2 = randi(n);   % 产生 1-n 中的一个随机整数
        path1 = path0;  % 将 path0 的值给 path1
        path1(c1) = path0(c2);  %改 path1 第 c1 个位置的元素为 path0
第 c2 个位置的元素
        path1(c2) = path0(c1);  %改 path1 第 c2 个位置的元素为 path0
第 c1 个位置的元素
    % 方法二
    elseif r < p1+p2 % 使用移位法产生新路径
        c1 = randi(n);   % 产生 1-n 中的一个随机整数
        c2 = randi(n);   % 产生 1-n 中的一个随机整数
        c3 = randi(n);   % 产生 1-n 中的一个随机整数
        sort_c = sort([c1 c2 c3]);  % 对 c1 c2 c3 从小到大进行排序
        c1 = sort_c(1);  c2 = sort_c(2);  c3 = sort_c(3);  % c1 <
= c2 <=  c3
        tem1 = path0(1:c1-1);
        tem2 = path0(c1:c2);
        tem3 = path0(c2+1:c3);
        tem4 = path0(c3+1:end);
        path1 = [tem1 tem3 tem2 tem4];
    % 方法三
    else  % 使用倒置法产生新路径
        c1 = randi(n);   % 产生 1-n 中的一个随机整数
        c2 = randi(n);   % 产生 1-n 中的一个随机整数
        if c1>c2  % 若 c1 比 c2 大，就交换 c1 和 c2 的值
            tem = c2;
            c2 = c1;
            c1 = tem;
        end
        tem1 = path0(1:c1-1);
        tem2 = path0(c1:c2);
        tem3 = path0(c2+1:end);
        path1 = [tem1 fliplr(tem2) tem3];   %矩阵的左右对称翻转
fliplr，上下对称翻转 flipud
    end
end
function  result =  jlh(path,d) % 定义函数算距离和
% 输入：path:旧路径（1 至 n 的一个序列），d：生成距离矩阵
    n = length(path);
    result = 0; % 初始化该路径走的距离为 0
    for i = 1:n-1
        result = d(path(i),path(i+1)) + result;  % 需要按照这个序
列更新走过的路程这个值
    end
    result = (d(path(1),path(n)) + result)*0.82808 - 5.173;  % 加
上从最后一个点返回到最开始那个点的距离
end
```

附录 6

介绍：该代码是 Matlab 编写的，作用是求解 N4 最优切割路径

```
clc;
clear;
%% 模拟退火算法——钢材切割问题
% 根据 tuoyuan.m 文件求解到椭圆最短距离点得到圆周上的点 x_m 和 y_mo
x_m = 66.0560825404235;
y_m = 21.7025143554360;
x_y = [100,0; 100,80; 85,80; 85,78; 75,80; 75,65; 70,65; 70,80;
60,80; 60,78; 50,80; 50,65; 45,65; 45,80; 35,80; 35,78; 25,80;
25,65; 20,65; 20,80; 10,80; 10,78; 0,80; 0,55; 8,55; 8,50; 0,50;
0,30; 8,30; 8,25;0,25; 0,0; 20,0; 20,15; 25,15; 25,0; 45,0; 45,15;
50,15; 50,0; 70,0; 70,15; 48,27; 57,27; 57,37; 60,37; 60,34;
73,34; 73,40; 60,40; 60,39; 57,39; 57,40; 48,40; 48,27; 48,52;
57,52; 57,46; 60,46; 60,54; 69,54; 69,43; 60,43; 60,44; 57,44;
57,43; 48,43; 48,52; x_m,y_m; x_m,y_m; 70,15; 75,15; 75,0; 85,80;
60,80; 35,80; 10,80]; % 输入笛卡尔坐标系位置
n = size(x_y,1);  % 点的数目

figure  % 新建 1 个图形窗口
% 画出每个特征点的分布散点图(特征点即顶点和拐点)
plot(x_y(:,1),x_y(:,2),'o');
hold on
d = zeros(n);   % 初始化所有城市的距离矩阵 d 全为 0
for i = 2:n  % 外循环 i 从 2 开始是因为城市 1 与城市 1 距离为 0，而后算下三角矩阵再对称即可得到距离矩阵
    for j = 1:i  % 内循环 j 得到每个城市之间的坐标和距离
        x_y_i = x_y(i,:);   x_i = x_y_i(1);     y_i = x_y_i(2);  % 城市 i 的横坐标为 x_i 而纵坐标为 y_i
        x_y_j = x_y(j,:);   x_j = x_y_j(1);     y_j = x_y_j(2);  % 城市 j 的横坐标为 x_j 而纵坐标为 y_j
        d(i,j) = sqrt((x_i-x_j)^2 + (y_i-y_j)^2);   % 算出城市 i 与 j 的距离
    end
end
% 切割线部分的路径值设为 0(必须放在下三角矩阵中，即 i>j)
d(2,1)=0;d(3,2)=0;d(4,3)=2;d(74,4)=2;d(74,5)=0;d(6,5)=0;d(7,6
)=0;d(8,7)=0;d(9,8)=0;d(10,9)=2;d(75,10)=2;
d(75,11)=0;d(12,11)=0;d(13,12)=0;d(14,13)=0;d(15,14)=0;d(16,1
5)=2;d(76,16)=2;d(76,17)=0;d(18,17)=0;
d(19,18)=0;d(20,19)=0;d(21,20)=0;d(22,21)=2;d(77,22)=2;d(77,2
3)=0;d(24,23)=0;d(25,24)=0;d(26,25)=0;
d(27,26)=0;d(28,27)=0;d(29,28)=0;d(30,29)=0;d(31,30)=0;d(32,3
1)=0;d(33,32)=0;d(34,33)=0;d(35,34)=0;
d(36,35)=0;d(37,36)=0;d(38,37)=0;d(39,38)=0;d(40,39)=0;d(41,4
0)=0;d(42,41)=0;d(69,42)=7.776772056115416;
d(69,43)=0;d(44,43)=0;d(45,44)=0;d(46,45)=3;d(47,46)=0;d(48,4
7)=0;d(49,48)=0;d(50,49)=0;d(51,50)=0;
```

```
d(52,51)=3;d(53,52)=0;d(54,53)=0;d(55,54)=0;d(56,55)=0;d(57,5
6)=0;d(58,57)=0;d(59,58)=3;d(60,59)=0;
d(61,60)=0;d(62,61)=0;d(63,62)=0;d(64,63)=0;d(65,64)=3;d(66,6
5)=0;d(67,66)=0;d(68,67)=0;d(70,68)=0;
d(71,70)=7.77677205611541;d(72,71)=0;d(73,72)=0;d(73,1)=0;
% 防止自身相连
d(74,3)=1000;
d(75,9)=1000;
d(76,15)=1000;
d(77,21)=1000;
d(55,43)=1000;
d(68,56)=1000;
d(70,69)=1000;
d(71,42)=1000;
% 生成距离矩阵的上三角矩阵
d = d+d';
%% 参数初始化
T0 = 1000;   % 模拟退火模型初始温度
T = T0; % 随着迭代变化，温度也会发生改变，第一次迭代时温度为 T0
maxgen = 1000;  % 设置最大迭代次数
Lk = 1000;  % 变化后不同温度下的迭代次数
beta = 0.95;  % 设置温度衰减系数
%%  随机生成初始解
rng('default');  % 将随机数种子设置为默认值，以确保每次运行时都生成相同
的随机序列
rng(40);  % 设置随机数种子为 40
path0 = randperm(n);  % 随机生成一个 1-n 的序列作为初始的路径 path0，
随后不断改进
best_path=path0; % 默认初始路径为最优路径
result0 = jlh(path0,d); % 调用 jlh 函数计算当前路径的距离，随后不断改
进
%% 定义中间变量，便于输出结果和绘图
min_result = result0;     % 初始化路径下对应的距离为 result0
RESULT = zeros(maxgen,1); % 每一次外层循环结束后找到的 min_result 然
后绘图观察某段时间后值最小且不变的情况即为所求优化解
%% 模拟退火过程
for iter = 1 : maxgen  % 外循环 iter，采用指定最大迭代次数
    for i = 1 : Lk  %  内循环，即在每个温度下开始迭代，整体共迭代 maxgen*Lk
次
        path1 = new_path(path0);  % 调用 new_path 函数生成新的路径
        result1 = jlh(path1,d); % 计算新路径下的距离
        % 条件判断 若新解距离短，则把新解的值赋值给原来的解
        if result1 < result0
            path0 = path1; % 更新当前路径为新路径
            result0 = result1; % 更新最短路径的和值
        else
```

```
            p = exp(-(result1 - result0)/T); % 根据 Metropolis 准则计算一个可接受概率
            if rand(1) < p   % 生成一个随机数和这个概率比较，如果该随机数小于这个概率则接受新解
                path0 = path1;  % 更新当前路径为新路径
                result0 = result1; % 更新最短的路径的和值
            end
        end
        % 判断是否要更新找到的最佳的解值
        if result0 < min_result  % 比较最短路径的和值，若当前解更好，则对其进行更新
            min_result = result0;  % 更新最小的距离数值
            best_path = path0;  % 更新为找到的最短路径
        end
    end
    RESULT(iter) = min_result; % 保存本轮外循环结束后找到的最小距离在 RESULT 里方便观察
    T = beta*T;   % 温度下降法则
end
targetValue=1;
index = find(best_path == targetValue, 1, 'first'); % 找序号为 5 的作为切割点
a=best_path(1,1:index-1);
b=best_path(index:end);
best_path=[b,a];
best_path = [best_path,best_path(1)];   % 在最短路径的最后面加入第一个点的元素(闭合的)
disp('最佳的方案是: '); disp(mat2str(best_path)) % 显示最佳路径(闭合的)
disp('此时最优值是: '); disp(min_result) % 显示最短路径的和值
n = n+1;  % 点的个数加一个(因为加了个闭合的重复的点)
for i = 1:n-1
    j = i+1;
    x_y_i = x_y(best_path(i),:);   x_i = x_y_i(1);     y_i = x_y_i(2); % x_i 为点坐标 x 轴位置 y_i 为点坐标 y 轴位置
    x_y_j = x_y(best_path(j),:);   x_j = x_y_j(1);     y_j = x_y_j(2); % 此为最优路线下后一个城市的坐标
    plot([x_i,x_j],[y_i,y_j],'-b')    % 不断画图，每两个点就作出一条线段，直到所有的点
    pause(0.01)  % 缓冲 0.01s 再画下一条线段
    hold on % 准备继续画图
end
%% 画出最短路径随迭代次数增加而变化的图形
figure
plot(1:maxgen,RESULT,'b-'); % 自变量为迭代次数 1:maxgen，因变量 RESULT
```

```
xlabel('迭代次数'); % x 轴名称
ylabel('最短路径的和值'); % y 轴名称
function path1 = new_path(path0) % path0: 代入原来的路径
    n = length(path0); % n 为点数量
    % 随机选择两种生成新路径的方法
    p1 = 0.33;  % 自行设置使用交换法产生新路径的概率
    p2 = 0.33;  % 自行设置使用移位法产生新路径的概率
    r = rand(1); % 生成一个[0 1]间均匀分布的随机数
    % 方法一
    if  r< p1    % 用交换法产生新路径
        c1 = randi(n);   % 产生 1-n 中的一个随机整数
        c2 = randi(n);   % 产生 1-n 中的一个随机整数
        path1 = path0;  % 将 path0 的值给 path1
        path1(c1) = path0(c2);  %改 path1 第 c1 个位置的元素为 path0
第 c2 个位置的元素
        path1(c2) = path0(c1);  %改 path1 第 c2 个位置的元素为 path0
第 c1 个位置的元素
    % 方法二
    elseif r < p1+p2 % 使用移位法产生新路径
        c1 = randi(n);   % 产生 1-n 中的一个随机整数
        c2 = randi(n);   % 产生 1-n 中的一个随机整数
        c3 = randi(n);   % 产生 1-n 中的一个随机整数
        sort_c = sort([c1 c2 c3]);  % 对 c1 c2 c3 从小到大进行排序
        c1 = sort_c(1);  c2 = sort_c(2);  c3 = sort_c(3);  % c1 <
= c2 <=  c3
        tem1 = path0(1:c1-1);
        tem2 = path0(c1:c2);
        tem3 = path0(c2+1:c3);
        tem4 = path0(c3+1:end);
        path1 = [tem1 tem3 tem2 tem4];
    % 方法三
    else  % 使用倒置法产生新路径
        c1 = randi(n);   % 产生 1-n 中的一个随机整数
        c2 = randi(n);   % 产生 1-n 中的一个随机整数
        if c1>c2  % 若 c1 比 c2 大，就交换 c1 和 c2 的值
            tem = c2;
            c2 = c1;
            c1 = tem;
        end
        tem1 = path0(1:c1-1);
        tem2 = path0(c1:c2);
        tem3 = path0(c2+1:end);
        path1 = [tem1 fliplr(tem2) tem3];   %矩阵的左右对称翻转
fliplr，上下对称翻转 flipud
    end
end
```

```
function  result =  jlh(path,d) % 定义函数算距离和
% 输入：path:旧路径（1 至 n 的一个序列），d：生成距离矩阵
    n = length(path);
    result = 0; % 初始化该路径走的距离为 0
    for i = 1:n-1
        result = d(path(i),path(i+1)) + result;  % 需要按照这个序列更新走过的路程这个值
    end
    result = d(path(1),path(n)) + result;  % 加上从最后一个点返回到最开始那个点的距离
end
```